AGES 5–10 YEARS

Python Programming for Kids

& Early Readers

25 fun modules that take you from your very first line of code to building your own game!



```
print("Hi!")
```

```
for i in range(3):
```

```
pets = ["cat"]
```

Hello, Grown-Ups! How to Use This Course



Ages 5–7

Read the slides together. Let your child press the keys and spot the colours in the code.



Ages 8–10

Children can read, type and run every example on their own, then try the challenges.



Type it, run it

Every code box shows exactly what appears when it runs. All examples work in Python 3.



Little and often

One module is about 30–45 minutes. Take breaks, and repeat modules if needed.



Quiz, then answers

Each module ends with a quiz. The very next slide shows the answers.



Challenges

Every recap slide has a challenge. There is no single right answer. Praise trying!

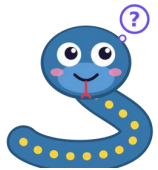
Meet Pip, Your Coding Buddy!



Pip is a friendly **python** snake who loves coding. Look for Pip's yellow speech bubbles with handy tips.

When Pip has a **?** bubble, Pip is thinking. Stop and think with Pip!

When Pip wears a **party hat**, you finished something. Celebrate!



How to Read a Code Slide

main.py

```
1 name = "Pip"
2 print("Hello,", name)
```

OUTPUT

```
Hello, Pip
```

The code box is where we type code. The numbers on the left are line numbers. You do not type them!

The output box shows what Python prints when you run the code.

CODE COLOURS HELP YOU READ

`print`	`"text"`	`42`	`if`	`# note`
commands	text (strings)	numbers	special words	comments

Your Coding Adventure Map

1. Hello, Computer!
2. Getting Python Ready
3. Your First Program
4. Python the Calculator
5. Variables: Name Boxes
6. Strings: Playing with Words
7. Talking to Your Program
8. Types of Data
9. True or False?
10. Making Choices with if
11. else and elif
12. and, or, not
13. for Loops: Repeat It!
14. while Loops
15. Loop Power-Ups
16. Lists: Collections
17. More List Magic
18. Tuples and Sets
19. Dictionaries
20. Functions: Your Own Commands
21. Functions That Give Back
22. Modules and Random
23. Turtle Drawing
24. Bugs and Errors
25. Grand Project: Build a Game

MODULE 1 OF 25

1 Hello, Computer!

Before we write code, let's meet the machine we will be talking to.

IN THIS MODULE YOU WILL:

- Find out what a computer really does
- Learn what a program is
- Meet Python, the friendly coding language



What Does a Computer Do?

1. A computer takes things **in**, **thinks** about them, then gives something **out**.
2. It is super fast. It can do millions of sums in one second!
3. But a computer is not clever on its own. It only does what we tell it.



IN

Keyboard, mouse, touch screen, microphone



THINK

The processor works out what to do



OUT

Screen, speakers, printer

A Program Is a List of Instructions

A **program** is like a recipe. The computer follows each step, in order, from top to bottom.

1. Get two slices of bread
2. Spread butter on one slice
3. Spread jam on top
4. Put the other slice on top
5. Eat and enjoy!



PIP SAYS

People who write programs are called **programmers** or **coders**. Soon that will be YOU!

Computers Need EXACT Instructions

A person can guess what you mean. A computer cannot guess at all.

A person hears...

"Make me a jam sandwich, please!"

A person can fill in the missing steps. They know what bread and jam are.

A computer needs...

Every single step, in the right order.

No spelling mistakes!

If a step is missing, the computer gets stuck.



PIP SAYS

Try it! Ask a grown-up to make a sandwich doing ONLY exactly what you say. It is very funny!

Meet Python!

1. Python is a **programming language**: a way to write instructions a computer understands.
2. It was made by **Guido van Rossum** and first shared in **1991**.
3. It is named after a funny TV show called **Monty Python**, not the snake!
4. Python reads a lot like English, so it is great for beginners.

```
print("Hi!")
```

This tiny line is real Python. It shows **Hi!** on the screen.

What Can You Build with Python?



Games

Make your own games with characters, scores and levels.



Robots

Control robots and gadgets, like a Raspberry Pi.



Science & Space

Scientists use Python to study space and nature.



Art

Draw colourful patterns with turtle graphics.



Websites & Apps

Big websites like Instagram use Python.



Smart Computers

Python helps build AI that can learn.

Lots of Languages, One Big Idea

1. People speak many languages. Computers have many languages too.
2. **Scratch** uses blocks you drag. Python uses words you type.
3. Once you learn one language, the next one is much easier!

Language	Good for
Scratch	Dragging blocks
Python	Typing real code
JavaScript	Websites
Swift	iPhone apps

Sneak Peek: Real Python Code!

Here is a real Python program. It has two lines. Each line is one instruction.

main.py

```
1 print("Hello, world!")
2 print("I am learning Python!")
```

OUTPUT

```
Hello, world!
I am learning Python!
```



PIP SAYS

Do not worry about how it works yet. In Module 3 you will write this yourself!

Quiz Time!

MODULE 1

Q1 What is a program?

- A) A list of instructions
- B) A type of snake
- C) A computer screen

Q2 Who decides what a computer does?

- A) The computer guesses
- B) The programmer
- C) The mouse

Q3 Python is named after...

- A) A big snake
- B) A comedy TV show
- C) A city

Quiz Time! — Answers

MODULE 1

Q1 What is a program?

A) A list of instructions

B) A type of snake

C) A computer screen

Why? A program is a list of instructions the computer follows in order, like a recipe.

Q2 Who decides what a computer does?

A) The computer guesses

B) The programmer

C) The mouse

Why? Computers only follow instructions. Programmers write those instructions.

Q3 Python is named after...

A) A big snake

B) A comedy TV show

C) A city

Why? Guido van Rossum named it after the comedy show Monty Python's Flying Circus.

Module 1 Recap & Challenge

WHAT WE LEARNED

- Computers take things in, think, then give things out.
- A program is a list of steps, like a recipe.
- Computers need exact instructions.
- Python is a programming language that reads a bit like English.

CHALLENGE TIME!

Write a 'program' for a friend on paper: the exact steps to draw a smiley face. Can they follow it without asking you anything?



MODULE 2 OF 25

2 Getting Python Ready

Let's get Python onto your computer and look around.

IN THIS MODULE YOU WILL:

- Install Python with a grown-up
- Explore IDLE, the Python Shell and Editor
- Save and run your very first file



What You Need

1. A computer with **Windows**, **macOS**, **Linux**, or a **Raspberry Pi**.
2. Python 3 from **python.org**. It is free!
3. A grown-up to help you install it.

A computer

A laptop or desktop works best.

Python 3

Free from python.org

A helper

A grown-up for the install



PIP SAYS

Raspberry Pi computers already have Python. You can skip the install!

Installing Python, Step by Step

Only download Python from the real website: **python.org**.

1. With a grown-up, open **python.org** in a web browser
2. Click **Downloads** and get the latest Python 3
3. Open the file. On Windows, tick **Add python.exe to PATH**
4. Click **Install Now** and wait
5. Find and open **IDLE** in your apps



PIP SAYS

On a Mac, the installer is a .pkg file. Just keep clicking **Continue** and then **Install**.

Meet IDLE

1. **IDLE** comes free with Python. It is where we will write and run code.
2. IDLE has two kinds of windows: the **Shell** and the **Editor**.
3. Another great choice for beginners is **Thonny** (from thonny.org).



This is the **prompt**. It means Python is ready and waiting for you!

Shell or Editor?

Both windows run Python. We use them for different jobs.

The Shell >>>

Type **one** line.

Press **Enter**.

Python answers straight away!

Great for quick tests.

It is not saved.

The Editor

Type **many** lines.

Save it as a **.py** file.

Press **F5** to run it.

Great for real programs.

You can open it again later!

Let's Play in the Shell!

Type each line after the >>> and press **Enter**. Python answers on the next line.

PYTHON SHELL

```
>>> 2 + 2
4
>>> "Hello!"
'Hello!'
>>> print("I am coding!")
I am coding!
>>> 10 * 10
100
```



PIP SAYS

Do not type the >>> yourself.
Python puts it there for you.

How to Save and Run a Program

MODULE 2

When a program is more than one line, use the Editor.

1. In IDLE, click **File → New File**
2. Type your code in the new window
3. Click **File → Save** and name it **hello.py**
4. Click **Run → Run Module** (or press **F5**)
5. Look in the Shell window to see the output!

Your First Saved Program

Type this into a new Editor window, save it as **hello.py**, then press **F5**.

hello.py

```
1 print("This is my first file!")
2 print("I saved it as hello.py")
```

OUTPUT

```
This is my first file!
I saved it as hello.py
```



PIP SAYS

Python files always end in **.py**. That is how the computer knows it is Python!

Coder's Keyboard Guide

MODULE 2

These keys will be your best friends.

Key	What it does	How to type it
`( )`	Brackets hold things for a command	Shift + 9 and Shift + 0
`"`	Quote marks wrap around text	Shift + 2 (UK keyboard)
`:`	A colon starts a block of code	Shift + ;
`Tab`	Makes a space at the start of a line	The Tab key
`Enter`	Starts a new line	The big Enter key

Quiz Time!

MODULE 2

Q1 What does >>> mean in the Shell?

- A) Fast forward
- B) Python is ready for you
- C) An error

Q2 What do Python file names end with?

- A) .doc
- B) .py
- C) .png

Q3 Which key runs a program in IDLE?

- A) F5
- B) Esc
- C) Space

Quiz Time! — Answers

MODULE 2

Q1 What does >>> mean in the Shell?

A) Fast forward

B) Python is ready for you

C) An error

Why? >>> is the prompt. It means Python is waiting for you to type.

Q2 What do Python file names end with?

A) .doc

B) .py

C) .png

Why? Python files end in .py, like hello.py.

Q3 Which key runs a program in IDLE?

A) F5

B) Esc

C) Space

Why? F5 is the shortcut for Run → Run Module.

Module 2 Recap & Challenge

WHAT WE LEARNED

- Python is free from python.org.
- IDLE has a Shell (one line at a time) and an Editor (whole programs).
- Save programs as .py files.
- Press F5 to run a program.

CHALLENGE TIME!

Open the Shell and try some sums of your own, like `5 + 5` and `100 - 1`. What is the biggest sum you can make?



MODULE 3 OF 25

3 Your First Program

Time to make the computer say hello!

IN THIS MODULE YOU WILL:

- Use print() to show messages
- Print words AND numbers
- Write comments, and fix your first bug



The print() Command

Every part of this line has a job. Let's look closely!

main.py

```
1 print("Hello, world!")
```

print is the command. It tells Python to show something.

( ) brackets hold what you want to show.

" " quote marks tell Python this is text.

Say Hello!

Each print() shows one line. Python runs the lines from top to bottom.

main.py

```
1 print("Hello, world!")
2 print("My name is Pip.")
3 print("I love Python!")
```

OUTPUT

```
Hello, world!
My name is Pip.
I love Python!
```



PIP SAYS

Change the words inside the quotes to make Python say anything you like!

Printing Numbers

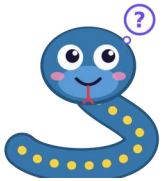
Numbers do not need quote marks. Look at the difference on line 3!

main.py

```
1 print(7)
2 print(3 + 4)
3 print("3 + 4")
```

OUTPUT

```
7
7
3 + 4
```



PIP SAYS

With quotes, "3 + 4" is just text, so Python prints it exactly. Without quotes, Python does the sum!

Printing Many Things at Once

Put a **comma** between things. Python adds a space between them for you.

main.py

```
print("I am", 8, "years old")
print("Cats have", 4, "legs")
```

OUTPUT

```
I am 8 years old
Cats have 4 legs
```



PIP SAYS

Commas go OUTSIDE the quote marks. They separate the things you want to print.

Draw a Picture with print()

MODULE 3

Text pictures are called **ASCII art**. Spaces and stars make a tree!

main.py

```
print("     *")
print("    ***")
print("   *****")
print("  *******")
print("    | |")
```

OUTPUT

```
     *
    ***
   *****
  *******
    | |
```



PIP SAYS

Try drawing a house or a rocket with letters and symbols!

Blank Lines with print()

An empty print() makes a blank line. It gives your output some space.

main.py

```
1 print("Roses are red")
2 print()
3 print("Python is fun")
```

OUTPUT

```
Roses are red

Python is fun
```

Comments: Notes for Humans

1. A comment starts with a **#** symbol.
2. Python **ignores** comments. They are notes for people reading the code.
3. Use comments to explain what your code does.

```
# hi!
```

Anything after **#** on a line is a comment.

Comments in Action

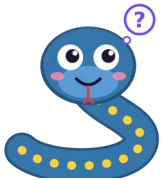
Only one line gets printed. Can you see why?

main.py

```
1 # My first program
2 print("Hello!")   # say hello
3 # print("This line is switched off")
```

OUTPUT

```
Hello!
```



PIP SAYS

Putting # at the start of a line switches it off. Coders call this **commenting out**.

Uh-Oh! Your First Bug

A mistake in code is called a **bug**. Here, one quote mark is missing.

main.py

```
1 print("Hello)
```



PIP SAYS

Text needs a quote mark at the start AND the end: "Hello"

OUTPUT

```
  File "main.py", line 1
    print("Hello)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

Capital Letters Matter!

Python is very fussy about capital letters. **Print** is not the same as **print**.

main.py

```
1 Print("Hello")
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    Print("Hello")
    ^^^^^
NameError: name 'Print' is not defined. Did you mean: 'print'?
```



PIP SAYS

Python even guesses what you meant! Look at the last line: **Did you mean: 'print'?**

Quiz Time!

MODULE 3

Q1 Which line prints Hi correctly?

A) `print(Hi)`

B) `print("Hi")`

C) `Print("Hi")`

Q2 What does `print(2 + 3)` show?

A) 2 + 3

B) 5

C) 23

Q3 What does Python do with a comment?

A) Prints it

B) Ignores it

C) Deletes it

Quiz Time! — Answers

MODULE 3

Q1 **Which line prints Hi correctly?**

A) `print(Hi)`

B) `print("Hi")`

C) `Print("Hi")`

Why? Text needs quote marks, and print must be all small letters.

Q2 **What does `print(2 + 3)` show?**

A) 2 + 3

B) 5

C) 23

Why? Without quote marks, Python works out the sum: 5.

Q3 **What does Python do with a comment?**

A) Prints it

B) Ignores it

C) Deletes it

Why? Comments start with # and Python skips them. They are notes for people.

Module 3 Recap & Challenge

WHAT WE LEARNED

- `print("text")` shows text on the screen.
- Numbers do not need quote marks.
- Commas print several things with spaces between.
- `#` starts a comment.
- A bug is a mistake. Error messages help you find it!

CHALLENGE TIME!

Write a program that prints your name, your age and your favourite food on three lines. Then add an ASCII art picture of something you love!



MODULE 4 OF 25

4 Python the Calculator

Python is brilliant at maths. Let's turn it into a super calculator!

IN THIS MODULE YOU WILL:

- Add, take away, times and divide
- Find remainders and powers
- Use brackets to choose what happens first



Python's Maths Symbols

Computers use * for times, because **x** is a letter.

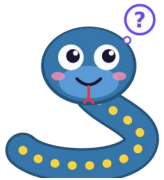
Symbol	Name	Example	Answer
+	Add	`5 + 3`	`8`
-	Take away	`9 - 4`	`5`
*	Times (multiply)	`6 * 7`	`42`
/	Divide	`20 / 4`	`5.0`
//	Whole-number divide	`17 // 5`	`3`
%	Remainder	`17 % 5`	`2`
**	Power	`2 ** 3`	`8`

Sums in the Shell

Open the Shell and try these. Press Enter after each one.

PYTHON SHELL

```
>>> 5 + 3
8
>>> 10 - 4
6
>>> 6 * 7
42
>>> 20 / 4
5.0
>>> 7 / 2
3.5
```

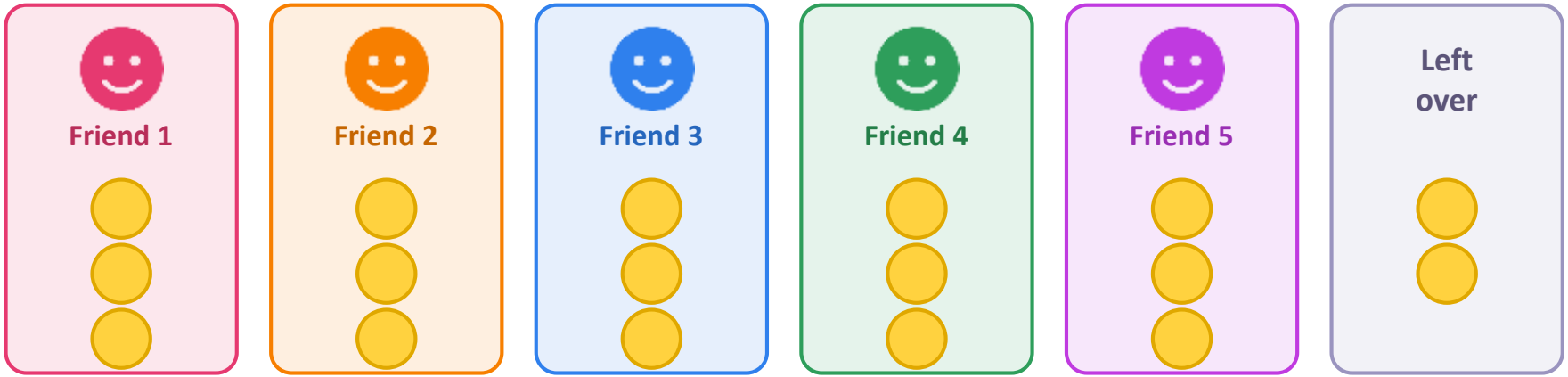


PIP SAYS

Did you spot it? Dividing with / always gives a **decimal** answer, like 5.0

Sharing Sweets: // and %

We have **17** sweets to share fairly between **5** friends.



`17 // 5 → 3`

sweets EACH friend gets

`17 % 5 → 2`

sweets LEFT OVER

Remainders and Powers

// gives the whole number answer. **%** gives what is left over.

PYTHON SHELL

```
>>> 17 // 5
3
>>> 17 % 5
2
>>> 2 ** 3
8
>>> 10 ** 2
100
```



PIP SAYS

`2 ** 3` means 2 × 2 × 2, which is 8. Powers grow fast!

Which Happens First?

Python follows the same order as school maths (**BIDMAS**): Brackets, Indices, Division and Multiplication, Addition and Subtraction.

No brackets

```
2 + 3 * 4
```

Times happens first:

3 * 4 = 12

then 2 + 12 = **14**

With brackets

```
(2 + 3) * 4
```

Brackets happen first:

2 + 3 = 5

then 5 * 4 = **20**

Check It with Python

Let's prove the last slide was right!

main.py

```
1 print(2 + 3 * 4)
2 print((2 + 3) * 4)
```

OUTPUT

```
14
20
```



PIP SAYS

When you are not sure, use brackets. They make your sums easier to read too.

Fun Facts Calculator

MODULE 4

Mix words and sums in one print() using commas.

main.py

```
print("Minutes in a day:", 24 * 60)
print("Seconds in an hour:", 60 * 60)
print("Legs on 3 spiders:", 3 * 8)
print("Days in 10 years:", 365 * 10)
```

OUTPUT

```
Minutes in a day: 1440
Seconds in an hour: 3600
Legs on 3 spiders: 24
Days in 10 years: 3650
```



PIP SAYS

Python does the sum first, then prints the answer next to the words.

Big Number Power!

MODULE 4

Python can handle really, really big numbers.

main.py

```
1 print(99 * 99)
2 print(2 ** 50)
3 print(123456789 * 9)
```

OUTPUT

```
9801
1125899906842624
1111111101
```



PIP SAYS

Try `print(2 ** 100)`. Python can work with numbers bigger than you can count!

Oops! Dividing by Zero

You cannot share 10 sweets between 0 friends. Python cannot do it either!

main.py

```
1 print(10 / 0)
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```



PIP SAYS

ZeroDivisionError is Python's way of saying: you cannot divide by zero.

Quiz Time!

MODULE 4

Q1 Which symbol means times in Python?

- A) x
- B) *
- C) %

Q2 What is `17 % 5`?

- A) 3
- B) 2
- C) 3.4

Q3 What does `(1 + 2) * 3` give?

- A) 7
- B) 9
- C) 6

Quiz Time! — Answers

MODULE 4

Q1 **Which symbol means times in Python?**

A) x

B) * ✓

C) %

Why? Python uses * (star) for times.

Q2 **What is `17 % 5`?**

A) 3

B) 2 ✓

C) 3.4

Why? % gives the remainder. 17 shared by 5 is 3 each with 2 left over.

Q3 **What does `(1 + 2) * 3` give?**

A) 7

B) 9 ✓

C) 6

Why? Brackets go first: 1 + 2 = 3, then 3 * 3 = 9.

Module 4 Recap & Challenge

WHAT WE LEARNED

- `+` `-` `*` `/` add, take away, times and divide.
- `/` always gives a decimal answer.
- `//` is whole-number divide and `%` is the remainder.
- `**` is power.
- Brackets go first!

CHALLENGE TIME!

How many hours have you been alive? Use Python: your age × 365 × 24. Then work out minutes too!



MODULE 5 OF 25

5 Variables: Name Boxes

Variables are how programs remember things.

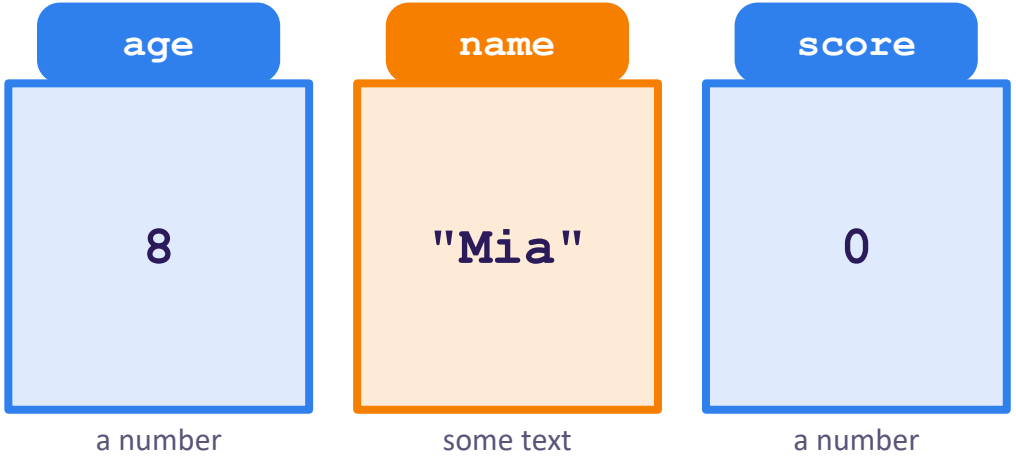
IN THIS MODULE YOU WILL:

- Store values in variables
- Change what is inside a variable
- Pick good names for your variables



A Variable Is a Labelled Box

1. A **variable** is like a box with a name label on it.
2. You can put a value inside, like a number or some text.
3. Later, use the name to find what is inside!



How to Make a Variable

In Python, = does not mean 'equals'. It means **store this**!

main.py

```
1 score = 10
```

score is the name label on the box.

= means: put the value on the right into the box on the left.

10 is the value that goes in the box.

Using Variables

Make variables first, then use their names in print().

main.py

```
1 name = "Mia"
2 age = 8
3 print(name)
4 print(age)
5 print(name, "is", age)
```

OUTPUT

```
Mia
8
Mia is 8
```



PIP SAYS

No quote marks around a variable name! Python looks inside the box instead.

Changing What Is in the Box

A variable can change. That is why it is called a **variable**: it can vary!

main.py

```
1 score = 0
2 print(score)
3 score = 5
4 print(score)
5 score = score + 10
6 print(score)
```

OUTPUT

```
0
5
15
```



PIP SAYS

When you put something new in the box, the old value is thrown away.

How score = score + 10 Works

Python always works out the **right side** first, then stores the answer on the left.

1

Look in the **score** box. It has **5**.

2

Work out **5 + 10**. That makes **15**.

3

Put **15** back into the **score** box.



PIP SAYS

This is how games keep score! Every time you win points, the score box gets bigger.

Handy Shortcuts: += and -=

coins += 5 is a short way to write **coins = coins + 5**.

main.py

```
1 coins = 10
2 coins += 5
3 print(coins)
4 lives = 3
5 lives -= 1
6 print(lives)
```

OUTPUT

```
15
2
```



PIP SAYS

There is also *= for times and /= for divide.

Naming Rules

Names can use letters, numbers and _ (underscore). They cannot start with a number.

Good names	Not allowed
`my_pet`	`2cool` starts with a number
`score2`	`my pet` has a space
`highScore`	`best-score` has a dash
`total_sweets`	`for` is a special Python word



PIP SAYS

Pick names that explain what is inside, like **lives** instead of **x**. And remember: **Age** and **age** are different names!

Variables Help Solve Problems

Name your numbers so the code tells a story.

main.py

```
1 sweets = 20
2 friends = 4
3 each = sweets // friends
4 print("Each friend gets", each)
```

OUTPUT

```
Each friend gets 5
```



PIP SAYS

Change sweets to 30. Run it again. The program works it out for you!

Bug: The Box Does Not Exist!

We made a box called **color**, then asked for **colour**. Those are different names!

main.py

```
1 color = "blue"
2 print(colour)
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(colour)
          ^^^^^^
NameError: name 'colour' is not defined. Did you mean: 'color'?
```



PIP SAYS

NameError means Python cannot find a box with that name. Check your spelling!

Quiz Time!

MODULE 5

Q1 What does = do in `x = 5`?

- A) Checks if x is 5
- B) Stores 5 in x
- C) Prints 5

Q2 Which is a good variable name?

- A) `my score`
- B) `2fast`
- C) `my_score`

Q3 `a = 3` then `a = a + 2`. What is in a?

- A) 3
- B) 5
- C) 32

Quiz Time! — Answers

MODULE 5

Q1 **What does = do in `x = 5`?**

A) Checks if x is 5

B) Stores 5 in x

C) Prints 5

Why? In Python, = means store the value on the right in the variable on the left.

Q2 **Which is a good variable name?**

A) `my score`

B) `2fast`

C) `my_score`

Why? Names cannot have spaces or start with a number. Underscores are fine.

Q3 **`a = 3` then `a = a + 2`. What is in a?**

A) 3

B) 5

C) 32

Why? Python works out a + 2 = 5, then stores 5 in a.

Module 5 Recap & Challenge

WHAT WE LEARNED

- A variable is a named box that stores a value.
- `=` means store, not equals.
- Variables can change: `score = score + 1`.
- `+=` and `-=` are shortcuts.
- Names: letters, numbers, _ and no spaces.

CHALLENGE TIME!

Make variables for your `pet_name`, `pet_legs` and `pet_age`. Print a sentence about your pet. Then change one value and run it again!



MODULE 6 OF 25

6 Strings: Playing with Words

In Python, text is called a string. Let's have fun with words!

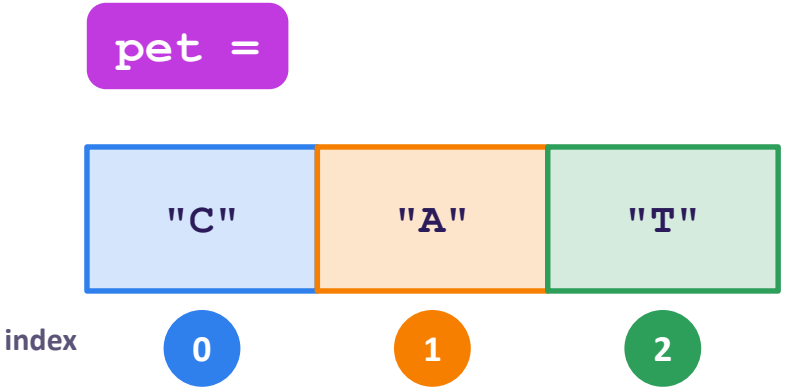
IN THIS MODULE YOU WILL:

- Join and repeat strings
- Count letters and change capitals
- Pick out letters with index numbers
- Fill in blanks with f-strings



What Is a String?

1. A **string** is text: letters, numbers, spaces and symbols.
2. Strings go inside quote marks, like `"Hello"` or `'Hello'`.
3. Think of letters **strung** together like beads on a string!



Single or Double Quotes

Both kinds of quote marks work. Just use the same kind at each end.

main.py

```
1 print('Hello!')
2 print("Hello!")
3 print("It's sunny today")
```

OUTPUT

```
Hello!
Hello!
It's sunny today
```



PIP SAYS

Use double quotes when your words have an apostrophe in them, like **It's**.

Joining Strings with +

Adding strings sticks them together. Coders call this **joining**.

main.py

```
1 first = "ice"
2 second = "cream"
3 print(first + second)
4 print(first + " " + second)
```

OUTPUT

```
icecream
ice cream
```



PIP SAYS

+ does not add a space for you. Put one in yourself with " ".

Repeating Strings with *

MODULE 6

Times a string by a number to repeat it.

main.py

```
1 print("ha" * 3)
2 print("=" * 20)
3 print("Hip hip hooray! " * 2)
```

OUTPUT

```
hahaha
====================
Hip hip hooray! Hip hip hooray!
```



PIP SAYS

`"="  *  20` is a quick way to draw a line under a title!

How Long Is It? len()

len() counts how many characters are in a string.

main.py

```
1 word = "elephant"
2 print(len(word))
3 print(len("hi there"))
```

OUTPUT

```
8
8
```



PIP SAYS

Spaces count too! "hi there" has 8 characters: 7 letters and 1 space.

Changing Capitals

MODULE 6

Strings have built-in tools called **methods**. Use a dot to reach them.

main.py

```
1 name = "pip the python"
2 print(name.upper())
3 print(name.title())
4 print(name.replace("python", "snake"))
```

OUTPUT

```
PIP THE PYTHON
Pip The Python
pip the snake
```

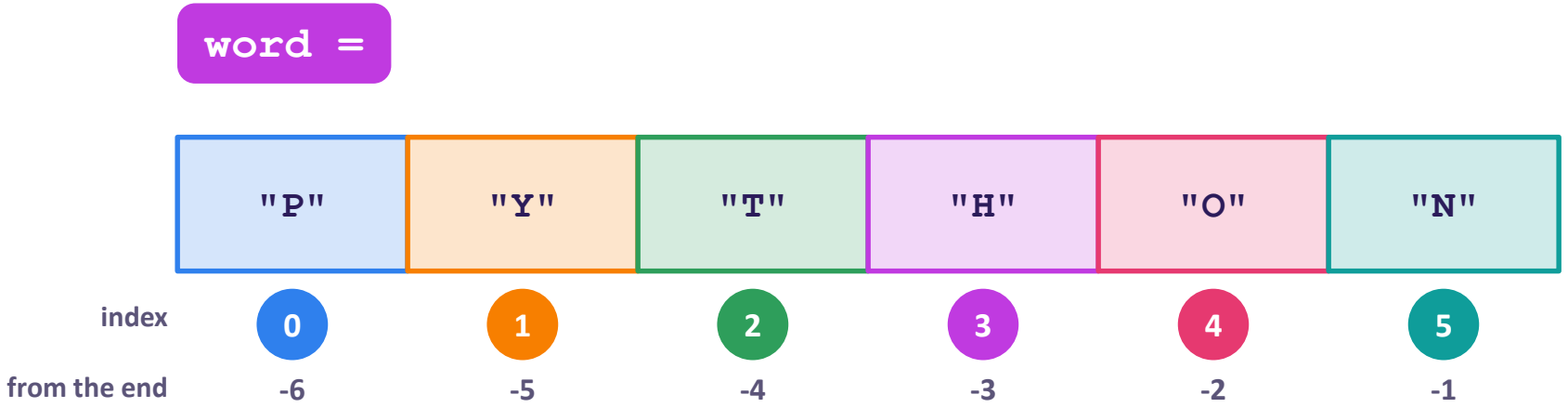


PIP SAYS

.lower() does the opposite of .upper(). Try it!

Every Letter Has a Number

Each character has an **index** number. Python starts counting at **0**, not 1!





PIP SAYS

Negative numbers count from the end. -1 is always the last letter.

Picking Out Letters

Put the index number in square brackets [ ] to get one letter.

main.py

```
1 word = "PYTHON"
2 print(word[0])
3 print(word[3])
4 print(word[-1])
```

OUTPUT

```
P
H
N
```



PIP SAYS

word[0] is the FIRST letter.
word[-1] is the LAST letter.

Slicing: Cut Out a Piece

word[start:stop] gives the letters from start up to (but not including) stop.

main.py

```
1 word = "RAINBOW"
2 print(word[0:4])
3 print(word[4:7])
4 print(word[4:])
```

OUTPUT

```
RAIN
BOW
BOW
```



PIP SAYS

Leave out the stop number to go all the way to the end, like word[4:].

f-Strings: Fill in the Blanks

Put an **f** before the quotes. Then anything in { } curly brackets gets filled in!

main.py

```
1 name = "Zara"
2 age = 7
3 print(f"{name} is {age} years old.")
4 print(f"Next year, {name} will be {age + 1}.")
```

OUTPUT

```
Zara is 7 years old.
Next year, Zara will be 8.
```



PIP SAYS

You can even do sums inside the curly brackets, like {age + 1}.

Bug: Mixing Words and Numbers

You cannot use + to join a string and a number.

main.py

```
1 age = 8
2 print("I am " + age)
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print("I am " + age)
          ~~~~~~~~^~~~
TypeError: can only concatenate str (not "int") to str
```



PIP SAYS

Fix it with commas: print("I am", age), or with an f-string: print(f"I am {age}")

Quiz Time!

Q1 What does `"ab" * 3` make?

- A) "ab3"
- B) "abab ab"
- C) An error

Q2 What is `len("cat")`?

- A) 3
- B) 2
- C) 4

Q3 If `word = "SNAKE"`, what is `word[0]`?

- A) "N"
- B) "S"
- C) "E"

Quiz Time! — Answers

Q1 What does `"ab" * 3` make?

A) "ab3"

B) "abab ab"

C) An error

Why? * repeats a string, so ab appears 3 times.

Q2 What is `len("cat")`?

A) 3

B) 2

C) 4

Why? len() counts characters. c, a, t is 3 characters.

Q3 If `word = "SNAKE"`, what is `word[0]`?

A) "N"

B) "S"

C) "E"

Why? Index 0 is always the first character.

Module 6 Recap & Challenge

WHAT WE LEARNED

- Strings are text in quote marks.
- `+` joins strings and `*` repeats them.
- `len()` counts characters.
- `word[0]` is the first letter. Indexes start at 0!
- f-strings fill in blanks: `f"Hi {name}"`

CHALLENGE TIME!

Store your first name in a variable. Print it in capitals, print how many letters it has, and print its first and last letters!



MODULE 7 OF 25

7 Talking to Your Program

Programs get exciting when they talk to you!

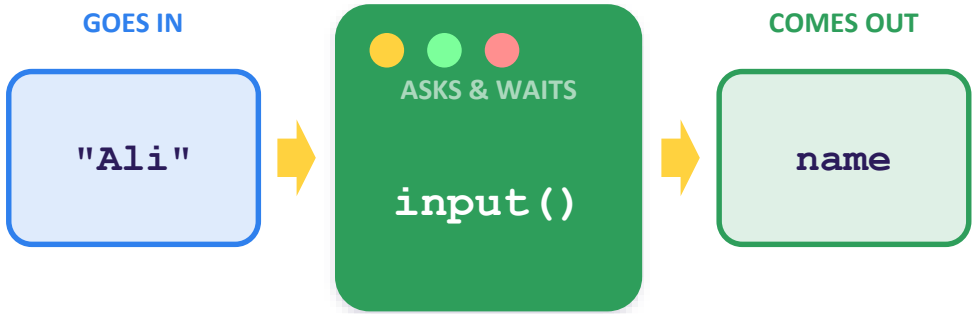
IN THIS MODULE YOU WILL:

- Ask questions with input()
- Use the answers in your program
- Turn typed answers into numbers



input() Asks a Question

1. **input()** shows a question and **waits** for you to type.
2. When you press Enter, your answer goes into a variable.
3. Now your program can use your answer!



You type **Ali**, it lands in the **name** box.

Hello, You!

Run this, type your name, then press Enter.

main.py

```
1 name = input("What is your name? ")
2 print("Hello,", name)
3 print("Nice to meet you!")
```

OUTPUT

```
What is your name? Ali
Hello, Ali
Nice to meet you!
```



PIP SAYS

In the output, the **yellow** words are what the person typed.

Inside an input() Line

Let's look at each part of this line.

main.py

```
1 name = input("What is your name? ")
```

name = is the box that catches the answer.

input( ) shows the question and waits.

The question goes inside quote marks.

A **space** before the last quote stops the answer touching the question.

Ask Two Questions

Each input() gets its own variable.

main.py

```
1 colour = input("Pick a colour: ")
2 animal = input("Pick an animal: ")
3 print(f"Imagine a {colour} {animal}!")
```

OUTPUT

```
Pick a colour: purple
Pick an animal: elephant
Imagine a purple elephant!
```



PIP SAYS

Run it again with different answers. Every run can be different!

Make a Silly Story

This is a word game. Ask for words, then put them in a story!

main.py

```
1 hero = input("Name a hero: ")
2 food = input("Name a food: ")
3 place = input("Name a place: ")
4 print(f"{hero} ate a giant {food}")
5 print(f"at the top of {place}!")
```

OUTPUT

```
Name a hero: Captain Cat
Name a food: pizza
Name a place: the moon
Captain Cat ate a giant pizza
at the top of the moon!
```



Watch Out! input() Gives a String

1. Everything you type comes back as a **string**, even numbers.
2. So if you type **8**, Python stores `"8"`, which is text.
3. To do maths, turn it into a number with **int()**.



PIP SAYS

int() turns a string like "8" into the number 8.

What you type	What Python stores
8	`"8"` (text, not a number!)

Bug: Adding to an Answer

We want to add 1 to the age. But age is a string!

main.py

```
1 age = input("How old are you? ")
2 print(age + 1)
```

OUTPUT

```
How old are you? 8
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(age + 1)
          ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```



PIP SAYS

Python cannot add the number 1 to the text "8". Let's fix it on the next slide.

Fixed with int()!

Wrap input() inside int(). Now age is a real number.

main.py

```
1 age = int(input("How old are you? "))
2 print("Next year you will be", age + 1)
```

OUTPUT

```
How old are you? 8
Next year you will be 9
```



PIP SAYS

Read it from the inside out: input() gets text, then int() turns it into a number.

A Tiny Calculator

Ask for two numbers, then do maths with them.

main.py

```
1 a = int(input("First number: "))
2 b = int(input("Second number: "))
3 print(a, "+", b, "=", a + b)
4 print(a, "x", b, "=", a * b)
```

OUTPUT

```
First number: 6
Second number: 7
6 + 7 = 13
6 x 7 = 42
```



PIP SAYS

Try adding lines for take away and divide too!

Chat with a Robot

MODULE 7

Use print() and input() together to make a chatty program.

robot.py

```
1 print("BEEP BOOP! I am Robo.")
2 name = input("What is your name? ")
3 food = input("What food do you like? ")
4 print(f"Hi {name}! I like {food} too!")
5 print("But robots only eat electricity.")
```

OUTPUT

```
BEEP BOOP! I am Robo.
What is your name? Sam
What food do you like? pasta
Hi Sam! I like pasta too!
But robots only eat electricity.
```



Quiz Time!

Q1 What does input() do?

A) Prints a picture

B) Waits for you to type

C) Deletes a variable

Q2 You type 5 into input(). What does Python store?

A) The number 5

B) The string `"5"`

C) Nothing

Q3 How do you turn the string "5" into a number?

A) `int("5")`

B) `str("5")`

C) `len("5")`

Quiz Time! — Answers

MODULE 7

Q1 What does input() do?

A) Prints a picture

B) Waits for you to type

C) Deletes a variable

Why? input() shows a question and waits for an answer.

Q2 You type 5 into input(). What does Python store?

A) The number 5

B) The string `"5"`

C) Nothing

Why? input() always gives back a string.

Q3 How do you turn the string "5" into a number?

A) `int("5")`

B) `str("5")`

C) `len("5")`

Why? int() turns a string of digits into a whole number.

Module 7 Recap & Challenge

WHAT WE LEARNED

- `input()` asks a question and waits.
- Store the answer in a variable.
- input() always gives a string.
- Use `int(input())` to get a number.

CHALLENGE TIME!

Make a 'Getting to Know You' program. Ask for a name, an age and a favourite animal. Then print a sentence using all three answers!



MODULE 8 OF 25

8 Types of Data

Python sorts data into different kinds, called types.

IN THIS MODULE YOU WILL:

- Meet int, float, str and bool
- Check the type of anything with type()
- Change one type into another



The Four Main Types

Every value in Python has a **type**. The type tells Python what it can do with the value.

int

Whole numbers, like **7**, **0** or **-3**

float

Numbers with a dot, like **3.5** or **0.25**

str

Text in quotes, like **"hello"**

bool

Only **True** or **False**



PIP SAYS

int is short for integer. **str** is short for string. **bool** is short for Boolean.

What Type Is It? type()

type() tells you what kind of data something is.

main.py

```
1 print(type(7))
2 print(type(3.5))
3 print(type("hi"))
4 print(type(True))
```

OUTPUT

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```



PIP SAYS

Python calls a type a **class**. So <class 'int'> means 'this is an int'.

7 and "7" Are Different!

Quote marks change everything!

7 (int)

A real number.

`7 + 1` gives **8**

`7 * 2` gives **14**

"7" (str)

Text that looks like a number.

`"7" + "1"` gives **"71"**

`"7" * 2` gives **"77"**



PIP SAYS

Python joins strings and adds numbers. Always check which one you have!

Proof: Numbers vs Strings

MODULE 8

Same characters, very different answers.

main.py

```
1 print(7 + 1)
2 print("7" + "1")
3 print(7 * 2)
4 print("7" * 2)
```

OUTPUT

```
8
71
14
77
```

Changing Types

int(), **float()** and **str()** turn a value into that type.

main.py

```
1 print(int("42") + 8)
2 print(str(42) + "!")
3 print(float("2.5"))
4 print(int(3.9))
```

OUTPUT

```
50
42!
2.5
3
```



PIP SAYS

int(3.9) gives 3, not 4. int() just chops off the decimal part!

Rounding Numbers

round() rounds to the nearest whole number, or to some decimal places.

main.py

```
1 print(round(3.7))
2 print(round(3.2))
3 print(round(2.567, 2))
```

OUTPUT

```
4
3
2.57
```



PIP SAYS

round(2.567, 2) keeps 2 numbers after the dot.

Shop Maths with Floats

MODULE 8

Money often has decimals, so we use floats.

main.py

```
price = 2.5
qty = 3
total = price * qty
print("Total:", total)
print(f"Total: £{total:.2f}")
```

OUTPUT

```
Total: 7.5
Total: £7.50
```



PIP SAYS

:.2f inside the curly brackets shows exactly 2 numbers after the dot, like real prices!

Wow Fact: Computer Fractions

Computers store decimals in a special way, so sometimes the answer is a tiny bit off!

main.py

```
1 print(0.1 + 0.2)
2 print(round(0.1 + 0.2, 2))
```

OUTPUT

```
0.30000000000000004
0.3
```



PIP SAYS

This happens in almost every programming language. round() tidies it up.

Bug: Not a Number!

int() can only turn digits into a number. "hello" has no digits!

main.py

```
1 number = int("hello")
```



PIP SAYS

ValueError means the value is the wrong kind for the job.

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    number = int("hello")
ValueError: invalid literal for int() with base 10: 'hello'
```

Type Cheat Sheet

Keep this slide handy!

Type	Example	Change into it with
`int`	`42`	`int("42")`
`float`	`3.14`	`float("3.14")`
`str`	`"hello"`	`str(42)`
`bool`	`True`	`bool(1)`

Quiz Time!

Q1 What type is `3.5`?

- A) int
- B) float
- C) str

Q2 What does `"3" + "4"` give?

- A) 7
- B) `"34"`
- C) An error

Q3 What does `int(9.8)` give?

- A) 10
- B) 9
- C) 9.8

Quiz Time! — Answers

Q1 **What type is `3.5`?**

A) int

B) float

C) str

Why? Numbers with a decimal point are floats.

Q2 **What does `"3" + "4"` give?**

A) 7

B) `"34"`

C) An error

Why? Both are strings, so + joins them.

Q3 **What does `int(9.8)` give?**

A) 10

B) 9

C) 9.8

Why? int() chops off the decimal part. It does not round.

Module 8 Recap & Challenge

WHAT WE LEARNED

- Four main types: `int`, `float`, `str`, `bool`.
- `type()` tells you the type.
- `7` and `"7"` are different!
- `int()`, `float()` and `str()` change types.
- `round()` rounds numbers.

CHALLENGE TIME!

Ask for the price of a toy and how many you want (use `float()` and `int()`). Print the total cost with £ and 2 decimal places!



MODULE 9 OF 25

9 True or False?

Computers love simple questions with yes or no answers.

IN THIS MODULE YOU WILL:

- Meet the Boolean values True and False
- Compare things with == and !=
- Use < and > to test numbers



Booleans: Yes or No

1. A **Boolean** can only be **True** or **False**.
2. It is like a light switch: on or off.
3. Always use a capital **T** and capital **F**!

True

Yes! On! Correct!

False

No! Off! Wrong!



PIP SAYS

Booleans are named after **George Boole**, a maths teacher born in 1815.

Comparison Symbols

A comparison asks a question. Python answers True or False.

Symbol	Means	Example	Answer
`==`	is equal to	`5 == 5`	`True`
`!=`	is not equal to	`5 != 3`	`True`
`>`	is bigger than	`2 > 9`	`False`
`<`	is smaller than	`2 < 9`	`True`
`>=`	is bigger than or equal to	`4 >= 4`	`True`
`<=`	is smaller than or equal to	`7 <= 3`	`False`

Ask Python Questions

Try each one in the Shell.

PYTHON SHELL

```
>>> 5 > 3
True
>>> 2 > 9
False
>>> 4 == 4
True
>>> 4 != 4
False
>>> "cat" == "cat"
True
>>> "Cat" == "cat"
False
```



PIP SAYS

"Cat" is not equal to "cat"! Capital letters make strings different.

= and == Are Different!

= (one equals)

Stores a value.

```
x = 5
```

Put 5 in the box called x.

== (two equals)

Asks a question.

```
x == 5
```

Is x the same as 5? True or False?



PIP SAYS

Say **gets** for = and **is equal to** for ==. That helps you remember!

Comparing Variables

Comparisons work with variables too.

main.py

```
1 age = 9
2 print(age >= 8)
3 print(age < 5)
4 print(age == 9)
```

OUTPUT

```
True
False
True
```

Store a Boolean in a Variable

True and False can go in a box, just like numbers and words.

main.py

```
1 is_raining = False
2 has_umbrella = True
3 print(is_raining)
4 print(type(has_umbrella))
```

OUTPUT

```
False
<class 'bool'>
```



PIP SAYS

Notice: no quote marks! "True" in quotes would be a string, not a Boolean.

Password Checker

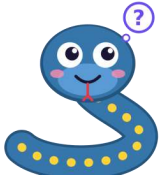
Compare what someone typed with the secret word.

main.py

```
1 guess = input("Password? ")
2 print(guess == "banana")
```

OUTPUT

```
Password? apple
False
```



PIP SAYS

Run it again and type banana. What do you get?

Is It Inside? The in Word

in checks if one string is inside another.

main.py

```
1 print("a" in "cat")
2 print("dog" in "hotdog")
3 print("z" in "zebra")
4 print("x" in "apple")
```

OUTPUT

```
True
True
True
False
```



PIP SAYS

in gives True or False, just like == does.

Comparing Words and Decimals

Strings are compared in alphabet order. Ints and floats can be compared too.

PYTHON SHELL

```
>>> "apple" < "banana"
True
>>> "b" > "a"
True
>>> 10 > 9.5
True
>>> 3 == 3.0
True
```



PIP SAYS

3 == 3.0 is True. The value is the same, even though the types are different!

Quiz Time!

Q1 What does `10 > 3` give?

- A) True
- B) False
- C) 10

Q2 Which one ASKS if x is 5?

- A) `x = 5`
- B) `x == 5`
- C) `x => 5`

Q3 What does `"Hi" == "hi"` give?

- A) True
- B) False
- C) An error

Quiz Time! — Answers

Q1 **What does `10 > 3` give?**

A) True

B) False

C) 10

Why? 10 is bigger than 3, so it is True.

Q2 **Which one ASKS if x is 5?**

A) `x = 5`

B) `x == 5`

C) `x => 5`

Why? == asks a question. = stores a value.

Q3 **What does `"Hi" == "hi"` give?**

A) True

B) False

C) An error

Why? Capital H and small h are different, so it is False.

Module 9 Recap & Challenge

WHAT WE LEARNED

- Booleans are `True` or `False`.
- `==` equal, `!=` not equal.
- `<` `>` `<=` `>=` compare sizes.
- `=` stores but `==` asks.
- `in` checks if something is inside a string.

CHALLENGE TIME!

Write a program that asks 'What is the capital of France?' and prints True if the answer is `"Paris"`. Can you make it work for `"paris"` too? (Hint: `.title()`)



MODULE 10 OF 25

10 Making Choices with if

Programs can make decisions, just like you do every day.

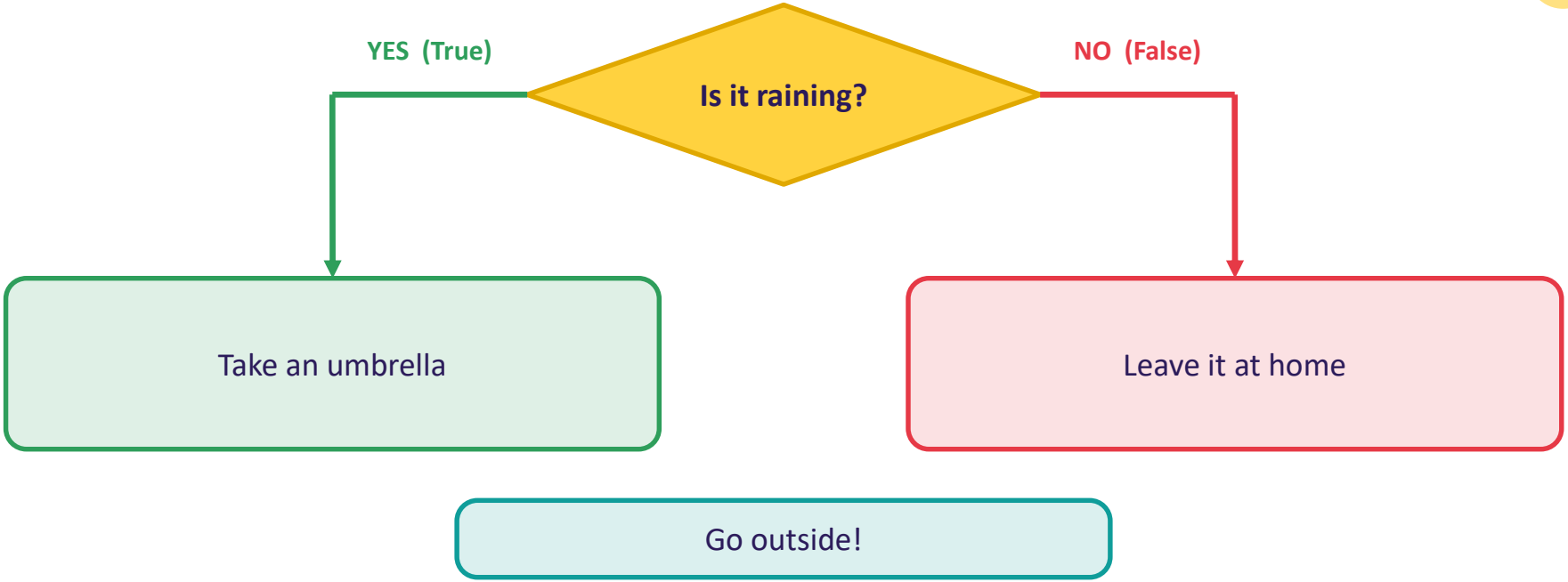
IN THIS MODULE YOU WILL:

- Run code only when something is True
- Use a colon and indentation
- Build a quiz question



Decisions in Real Life

If it is raining, take an umbrella. Computers decide things the same way.



Inside an if Statement

Every if has the same parts.

main.py

```
1 temperature = 30
2 if temperature > 25:
3     print("Wear shorts!")
```

if starts the decision.

temperature > 25 is the question. It must be True or False.

The **colon :** means: here comes the code to run.

4 spaces at the start. This line only runs if the answer is True.

When It Is True

30 is bigger than 25, so the indented lines run.

main.py

```
1 temperature = 30
2 if temperature > 25:
3     print("It is hot!")
4     print("Wear shorts!")
5 print("Have a nice day.")
```

OUTPUT

```
It is hot!
Wear shorts!
Have a nice day.
```



PIP SAYS

All the lines with the same indent belong to the if. They are called a **block**.

When It Is False

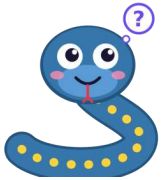
Now it is 10. The question is False, so the indented lines are **skipped**.

main.py

```
1 temperature = 10
2 if temperature > 25:
3     print("It is hot!")
4     print("Wear shorts!")
5 print("Have a nice day.")
```

OUTPUT

```
Have a nice day.
```



PIP SAYS

The last line has no indent, so it always runs.

Indentation: Inside or Outside?

In Python, spaces at the start of a line really matter!

Indented (inside)

4 spaces at the start.

Belongs to the if.

Runs **only** when the question is True.

Not indented (outside)

No spaces at the start.

Not part of the if.

Runs **every time**.



PIP SAYS

IDLE adds the 4 spaces for you after a colon. Press Backspace to go back out.

Quiz Question!

Check the player's answer with ==.

main.py

```
1 answer = input("What is 3 + 4? ")
2 if answer == "7":
3     print("Correct! Well done!")
4 print("Thanks for playing.")
```

OUTPUT

```
What is 3 + 4? 7
Correct! Well done!
Thanks for playing.
```



PIP SAYS

We compare with "7" in quotes, because input() gives a string.

Lots of Separate ifs

Python checks each if on its own. More than one can be True!

main.py

```
1 score = 85
2 if score > 50:
3     print("You passed!")
4 if score > 80:
5     print("Amazing score!")
6 if score == 100:
7     print("Perfect!")
```

OUTPUT

```
You passed!
Amazing score!
```



PIP SAYS

Change the score to 100 and see how many messages appear.

Using Numbers from input()

Turn the answer into a number, then compare it.

main.py

```
1 age = int(input("How old are you? "))
2 if age >= 8:
3     print("You can ride the big rollercoaster!")
4 print("Enjoy the theme park!")
```

OUTPUT

```
How old are you? 9
You can ride the big rollercoaster!
Enjoy the theme park!
```



Bug: Forgot the Colon

Every if line must end with a colon.

main.py

```
1 age = 9
2 if age > 8
3     print("Big kid!")
```

OUTPUT

```
  File "main.py", line 2
    if age > 8
             ^
SyntaxError: expected ':'
```



PIP SAYS

Python points right at the problem: **expected ':'**

Bug: Forgot to Indent

The line after a colon must be indented.

main.py

```
1 age = 9
2 if age > 8:
3 print("Big kid!")
```



PIP SAYS

IndentationError means the spaces at the start of a line are wrong.

OUTPUT

```
  File "main.py", line 3
    print("Big kid!")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 2
```

Quiz Time!

Q1 What goes at the end of an if line?

- **A)** A full stop .
- **B)** A colon :
- **C)** A comma ,

Q2 When does an indented line under an if run?

- **A)** Always
- **B)** Only if it is True
- **C)** Never

Q3 How many spaces are normally used to indent?

- **A)** 1
- **B)** 4
- **C)** 10

Quiz Time! — Answers

Q1 **What goes at the end of an if line?**

A) A full stop .

B) A colon :

C) A comma ,

Why? Every if line ends with a colon.

Q2 **When does an indented line under an if run?**

A) Always

B) Only if it is True

C) Never

Why? The block under if only runs when the condition is True.

Q3 **How many spaces are normally used to indent?**

A) 1

B) 4

C) 10

Why? Python coders use 4 spaces. IDLE does it for you.

Module 10 Recap & Challenge

WHAT WE LEARNED

- `if` runs code only when a question is True.
- The if line ends with a colon `:`.
- The code inside is indented by 4 spaces.
- Lines that are not indented always run.

CHALLENGE TIME!

Make a 'Secret Door' program. Ask for a password. If it is `"open sesame"`, print a welcome message and a picture made of print() lines!



MODULE 11 OF 25

11 else and elif

What happens when the answer is No? Let's find out!

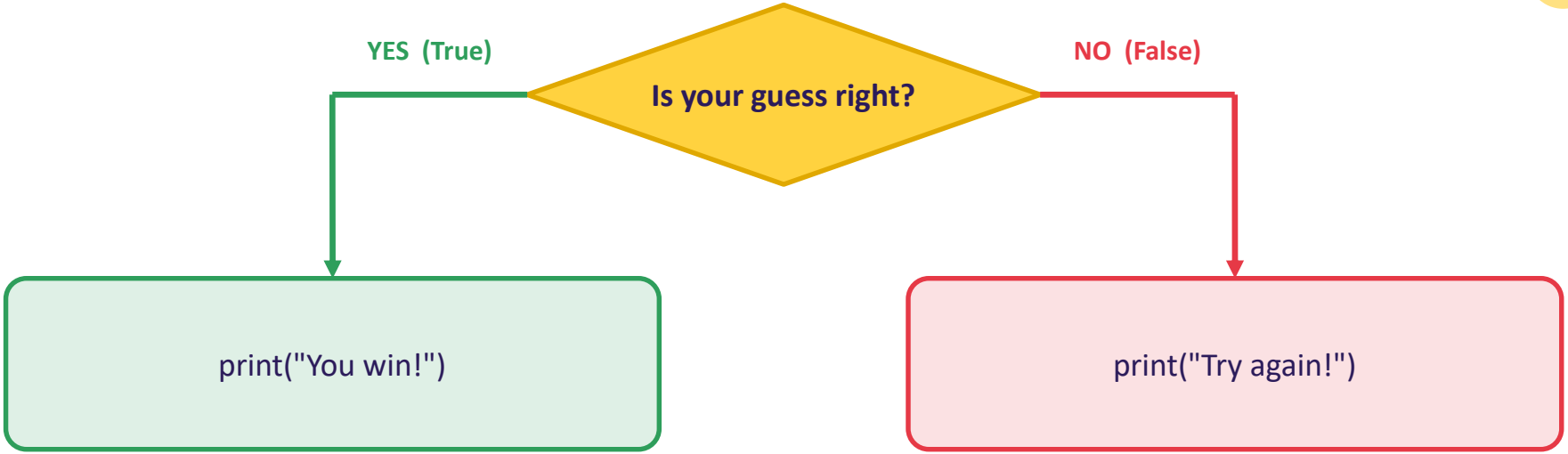
IN THIS MODULE YOU WILL:

- Choose between two paths with else
- Check many choices with elif
- Learn why the order of checks matters



Two Paths: if and else

else gives your program something to do when the question is False.



Exactly ONE path runs

Inside an if / else

else always comes straight after an if block.

main.py

```
1 age = 6
2 if age >= 8:
3     print("Big slide!")
4 else:
5     print("Little slide!")
```

The question is checked first.

This runs if the answer is **True**.

else: has no question. It needs a colon!

This runs if the answer is **False**.

if / else in Action

6 is not 8 or more, so the else block runs.

main.py

```
1 age = 6
2 if age >= 8:
3     print("Big slide!")
4 else:
5     print("Little slide!")
6 print("Have fun!")
```

OUTPUT

```
Little slide!
Have fun!
```



PIP SAYS

else lines up exactly under if. Both have no spaces before them.

Odd or Even?

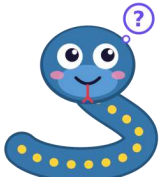
Even numbers have no remainder when you divide by 2. Remember %?

main.py

```
1 number = 7
2 if number % 2 == 0:
3     print(number, "is even")
4 else:
5     print(number, "is odd")
```

OUTPUT

```
7 is odd
```



PIP SAYS

Change the number to 10 and run it again.

elif: Another Question

1. **elif** is short for **else if**.
2. It asks a new question when the one before was False.
3. You can have as many elifs as you like!
4. Python stops at the **first** True answer.

1. `if light == "red":` Stop!
2. `elif light == "amber":` Get ready!
3. `elif light == "green":` Go!
4. `else:` Broken light!

Traffic Lights

Only ONE of these messages will ever be printed.

main.py

```
1 light = "amber"
2 if light == "red":
3     print("Stop!")
4 elif light == "amber":
5     print("Get ready...")
6 elif light == "green":
7     print("Go!")
8 else:
9     print("Broken light!")
```

OUTPUT

```
Get ready...
```



PIP SAYS

Try light = "blue". None of the questions are True, so else runs.

Star Award Checker

Checks go from the highest score down to the lowest.

main.py

```
score = 72
if score >= 90:
    print("Gold star!")
elif score >= 70:
    print("Silver star!")
elif score >= 50:
    print("Bronze star!")
else:
    print("Keep practising!")
```

OUTPUT

```
Silver star!
```



PIP SAYS

72 is not 90 or more, but it IS 70 or more. So we get Silver!

Order Matters!

This is a sneaky bug. 95 should get Gold! What went wrong?

main.py

```
1 score = 95
2 if score >= 50:
3     print("Bronze star!")
4 elif score >= 90:
5     print("Gold star!")
```

OUTPUT

```
Bronze star!
```



PIP SAYS

95 >= 50 is True, so Python stops there and never checks for Gold. Put the biggest test first!

What Should I Wear?

Use elif with input() to answer a question.

main.py

```
weather = input("Sunny, rainy or snowy? ")
if weather == "sunny":
    print("Sunglasses!")
elif weather == "rainy":
    print("Wellies!")
elif weather == "snowy":
    print("A big warm coat!")
else:
    print("I do not know that weather.")
```

OUTPUT

```
Sunny, rainy or snowy? rainy
Wellies!
```



An if Inside an if

You can put an if inside another if. This is called **nesting**.

main.py

```
1 age = 9
2 height = 125
3 if age >= 8:
4     if height >= 120:
5         print("You can ride!")
6     else:
7         print("Not tall enough yet.")
8 else:
9     print("Too young, sorry.")
```

OUTPUT

```
You can ride!
```



PIP SAYS

Look at the indents. The inner if has 8 spaces before its print.

Quiz Time!

Q1 When does the else block run?

A) Always

B) When the if is False

C) Never

Q2 What is elif short for?

A) else if

B) elf if

C) end if

Q3 How many blocks run in one if / elif / else?

A) All of them

B) Exactly one

C) Two

Quiz Time! — Answers

Q1 **When does the else block run?**

A) Always

B) When the if is False

C) Never

Why? else runs only when the if question is False.

Q2 **What is elif short for?**

A) else if

B) elf if

C) end if

Why? elif means else if: check another question.

Q3 **How many blocks run in one if / elif / else?**

A) All of them

B) Exactly one

C) Two

Why? Python runs the first True block, or else, and skips the rest.

Module 11 Recap & Challenge

WHAT WE LEARNED

- `else:` runs when the if question is False.
- `elif` asks another question.
- Python stops at the first True answer.
- Put the hardest check first!
- You can nest an if inside an if.

CHALLENGE TIME!

Make a 'Guess My Age' program. Ask for a guess and print `Too high!`, `Too low!` or `Correct!` using if, elif and else.



MODULE 12 OF 25

12 and, or, not

Some decisions need more than one question.

IN THIS MODULE YOU WILL:

- Join two questions with and
- Accept either answer with or
- Flip True and False with not



Three Logic Words

Python uses real English words to join questions together.



and

Both must be True. A sandwich needs bread AND a filling.



or

One True is enough. You can pay with cash OR card.



not

Flips the answer. not True is False.

The and Table

and is fussy. It is only True when **both** sides are True.

A	B	A and B	In words
`True`	`True`	`True`	both are True
`True`	`False`	`False`	one is False
`False`	`True`	`False`	one is False
`False`	`False`	`False`	both are False



PIP SAYS

Think of and as a door with two locks. You need BOTH keys to open it!

Using and

Both questions must be True to ride.

main.py

```
age = 9
height = 130
if age >= 8 and height >= 120:
    print("You can ride the big rollercoaster!")
else:
    print("Sorry, not this time.")
```

OUTPUT

```
You can ride the big rollercoaster!
```



The or Table

or is easy-going. It is True when **at least one** side is True.

A	B	A or B	In words
`True`	`True`	`True`	both are True
`True`	`False`	`True`	one is True
`False`	`True`	`True`	one is True
`False`	`False`	`False`	none are True



PIP SAYS

Think of or as a room with two doors. Either door gets you in!

Using or

Only one of the questions needs to be True.

main.py

```
1 day = "Saturday"
2 if day == "Saturday" or day == "Sunday":
3     print("Yay, it is the weekend!")
4 else:
5     print("It is a school day.")
```

OUTPUT

```
Yay, it is the weekend!
```



Using not

not flips True into False, and False into True.

main.py

```
1 is_raining = False
2 if not is_raining:
3     print("Let's play outside!")
4 else:
5     print("Let's build a den indoors.")
```

OUTPUT

```
Let's play outside!
```



PIP SAYS

Read it out loud: **if not raining**. It sounds just like English!

Try Logic in the Shell

Practise and, or and not on their own.

PYTHON SHELL

```
>>> True and False
False
>>> True or False
True
>>> not True
False
>>> 5 > 3 and 2 > 1
True
>>> 5 > 3 and 2 > 9
False
```

Is It Between 1 and 10?

Check that a number is inside a range. There is a neat shortcut on line 4!

main.py

```
1 number = 7
2 if number >= 1 and number <= 10:
3     print("In range!")
4 if 1 <= number <= 10:
5     print("Also in range!")
```

OUTPUT

```
In range!
Also in range!
```



PIP SAYS

1 <= number <= 10 reads like maths: is number between 1 and 10?

Sneaky Bug: The Always-True or

Monday is not the weekend, but Python says it is! Why?

main.py

```
1 day = "Monday"
2 if day == "Saturday" or "Sunday":
3     print("Weekend!")
```

OUTPUT

```
Weekend!
```



PIP SAYS

Python reads "Sunday" on its own as True. Always write the full question on both sides: day == "Sunday"

Secret Club Login

Both the username AND the code must be right.

login.py

```
1 user = input("Username: ")
2 code = input("Secret code: ")
3 if user == "pip" and code == "1234":
4     print("Welcome to the secret club!")
5 else:
6     print("Access denied!")
```

OUTPUT

```
Username: pip
Secret code: 1234
Welcome to the secret club!
```



Quiz Time!

Q1 What is `True and False`?

- A) True
- B) False
- C) Maybe

Q2 What is `False or True`?

- A) True
- B) False
- C) Error

Q3 What is `not False`?

- A) False
- B) True
- C) not

Quiz Time! — Answers

MODULE 12

Q1 What is `True and False`?

A) True | **B) False** | C) Maybe

Why? and needs both sides to be True.

Q2 What is `False or True`?

A) True | B) False | C) Error

Why? or only needs one side to be True.

Q3 What is `not False`?

A) False | **B) True** | C) not

Why? not flips the answer, so not False is True.

Module 12 Recap & Challenge

WHAT WE LEARNED

- `and`: both sides must be True.
- `or`: at least one side must be True.
- `not` flips True and False.
- Write the full question on both sides of `or`!

CHALLENGE TIME!

Make a 'Can I Have a Pet?' program. Ask if the room is tidy and if the homework is done (yes or no). Use `and` to decide!



MODULE 13 OF 25

13 for Loops: Repeat It!

Computers never get bored of repeating things. Loops let them!

IN THIS MODULE YOU WILL:

- Repeat code with a for loop
- Count with range()
- Loop over the letters in a string



Why Do We Need Loops?

Typing the same line again and again is boring. A **loop** repeats it for you!

Without a loop

```
print("Hooray!")
print("Hooray!")
print("Hooray!")
print("Hooray!")
print("Hooray!")
```

With a loop

```
for i in range(5):
    print("Hooray!")
```

Just 2 lines! And easy to change to 100!

Inside a for Loop

A for loop has a colon and an indent, just like if.

main.py

```
1 for i in range(3):
2     print("Hip hip hooray!")
```

for starts a loop.

i is a variable. It counts which turn we are on.

range(3) means: do it 3 times.

The indented code runs **once each time** round the loop.

Say It Five Times

The indented line runs 5 times. The last line runs once, after the loop.

main.py

```
1 for i in range(5):
2     print("I love Python!")
3 print("The loop is over.")
```

OUTPUT

```
I love Python!
I love Python!
I love Python!
I love Python!
I love Python!
The loop is over.
```

Watch the Counter

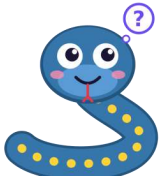
The loop variable changes every time round. Let's print it!

main.py

```
1 for number in range(5):
2     print(number)
```

OUTPUT

```
0
1
2
3
4
```



PIP SAYS

It counts 0, 1, 2, 3, 4. That is 5 numbers, but it starts at 0!

How range() Counts

range(5) gives 5 numbers, starting from 0. It stops BEFORE 5.

```
range(5)
```



Gives: 0, 1, 2, 3, 4



PIP SAYS

range always stops one before the end number. It is a bit like a slice!

Start and Stop: range(1, 6)

Give **two** numbers to choose where to start and where to stop.

```
range(1, 6)
```



Gives: 1, 2, 3, 4, 5

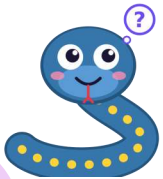
Jumping: range(0, 11, 2)

A **third** number is the step size. Here we jump by 2!

```
range(0, 11, 2)
```



Gives: 0, 2, 4, 6, 8, 10



PIP SAYS

These are the even numbers. Can you make range give the odd numbers?

Rocket Countdown

A negative step makes range count **backwards**.

main.py

```
for n in range(10, 0, -1):
    print(n)
print("Blast off!")
```



PIP SAYS

Start at 10, stop before 0, step -1. So it ends at 1!

OUTPUT

```
10
9
8
7
6
5
4
3
2
1
Blast off!
```

Loop Through a Word

A for loop can go through each letter in a string.

main.py

```
1 for letter in "PYTHON":
2     print(letter)
```

OUTPUT

```
P
Y
T
H
O
N
```



PIP SAYS

Each time round, letter holds the next character.

Times Table Machine

MODULE 13

One loop prints a whole times table!

main.py

```
1 n = 7
2 for i in range(1, 11):
3     print(f"{i} x {n} = {i * n}")
```



PIP SAYS

Change n to any number to get a new times table.

OUTPUT

```
1 x 7 = 7
2 x 7 = 14
3 x 7 = 21
4 x 7 = 28
5 x 7 = 35
6 x 7 = 42
7 x 7 = 49
8 x 7 = 56
9 x 7 = 63
10 x 7 = 70
```

Adding Up with a Loop

Start a total at 0, then add each number as the loop goes round.

main.py

```
1 total = 0
2 for n in range(1, 11):
3     total += n
4 print("1 + 2 + ... + 10 =", total)
```

OUTPUT

```
1 + 2 + ... + 10 = 55
```



PIP SAYS

The print is NOT indented, so it runs once at the end.

Quiz Time!

Q1 How many times does `for i in range(4):` loop?

- A) 3
- B) 4
- C) 5

Q2 What numbers does `range(2, 5)` give?

- A) 2, 3, 4
- B) 2, 3, 4, 5
- C) 3, 4, 5

Q3 What does a step of -1 do?

- A) Stops the loop
- B) Counts backwards
- C) Skips numbers

Quiz Time! — Answers

MODULE 13

Q1 How many times does `for i in range(4):` loop?

A) 3

B) 4

C) 5

Why? range(4) gives 0, 1, 2, 3. That is 4 numbers.

Q2 What numbers does `range(2, 5)` give?

A) 2, 3, 4

B) 2, 3, 4, 5

C) 3, 4, 5

Why? range starts at 2 and stops BEFORE 5.

Q3 What does a step of -1 do?

A) Stops the loop

B) Counts backwards

C) Skips numbers

Why? A negative step counts down.

Module 13 Recap & Challenge

WHAT WE LEARNED

- A `for` loop repeats code.
- `range(5)` gives 0 to 4.
- `range(start, stop, step)` gives you control.
- A for loop can go through letters in a string.
- Indented code is repeated. Unindented code is not.

CHALLENGE TIME!

Use a loop to print the 5 times table. Then print a countdown from 20 to 0 in steps of 5!



MODULE 14 OF 25

14 while Loops

A while loop keeps going until its question becomes False.

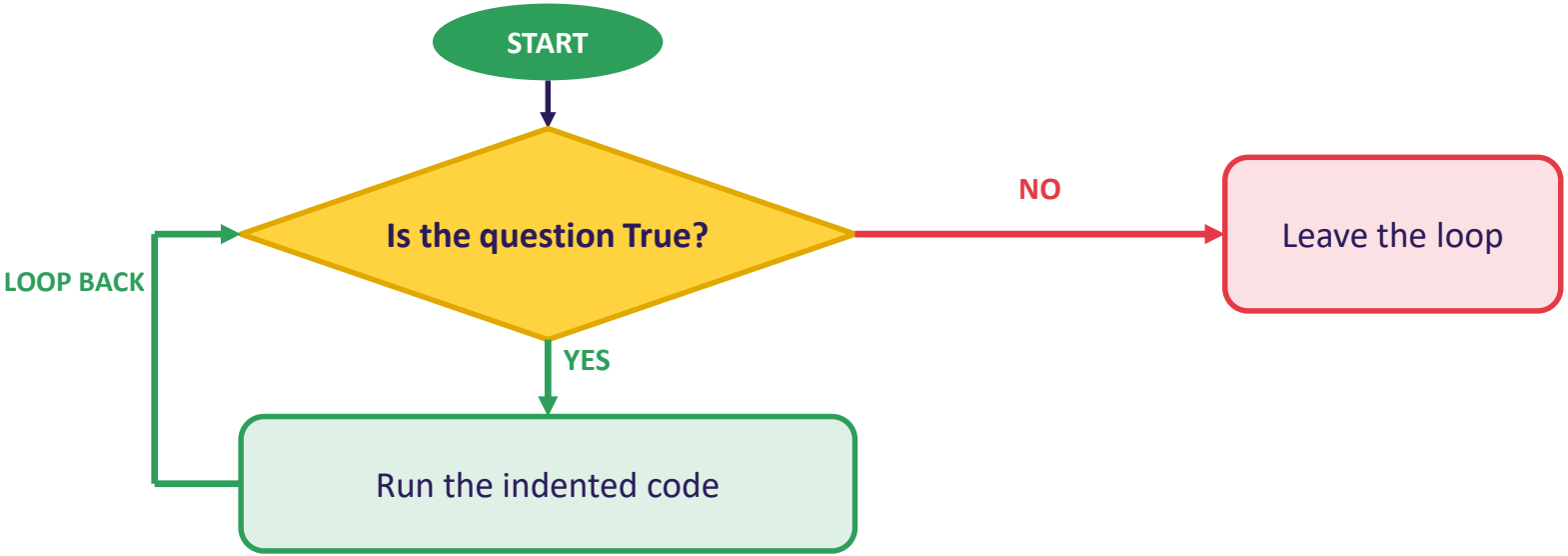
IN THIS MODULE YOU WILL:

- Repeat while something is True
- Keep asking until the answer is right
- Avoid loops that never stop



How a while Loop Works

Before every turn, Python checks the question. True means go round again!



Inside a while Loop

A while loop is like an if that keeps repeating.

main.py

```
1 count = 1
2 while count <= 3:
3     print(count)
4     count += 1
```

Make a variable **before** the loop.

while + a question + a colon.

This runs while the question is True.

Change the variable, or the loop never ends!

Counting with while

When count becomes 4, the question is False, so the loop stops.

main.py

```
1 count = 1
2 while count <= 3:
3     print(count)
4     count += 1
5 print("Done!")
```

OUTPUT

```
1
2
3
Done!
```

Countdown with while

This does the same job as the for loop countdown.

main.py

```
1 n = 5
2 while n > 0:
3     print(n)
4     n -= 1
5 print("Lift off!")
```

OUTPUT

```
5
4
3
2
1
Lift off!
```

Keep Asking!

The loop keeps going until the player gets it right.

main.py

```
guess = ""
while guess != "banana":
    guess = input("Guess my favourite fruit: ")
print("Yes! You got it!")
```

OUTPUT

```
Guess my favourite fruit: apple
Guess my favourite fruit: grape
Guess my favourite fruit: banana
Yes! You got it!
```



PIP SAYS

We start guess as an empty string "" so the question is True the first time.

Doubling Up

We do not know how many turns this needs. while is perfect for that!

main.py

```
1 n = 1
2 while n < 100:
3     print(n)
4     n = n * 2
```

OUTPUT

```
1
2
4
8
16
32
64
```



PIP SAYS

128 is not printed, because the loop stops when n is no longer less than 100.

Danger: The Never-Ending Loop!

1. If the question **never** becomes False, the loop goes on forever!
2. This is called an **infinite loop**.
3. To stop it, click the Shell and press **Ctrl + C**.



PIP SAYS

Always check that something inside your loop changes the question.

Never ends

```
n = 1

while n >
0:

print(n)
```

n never changes!

Stops

```
n = 5

while n >
0:

print(n)
    n -= 1
```

Are We There Yet?

A loop that repeats until you type **yes**.

main.py

```
answer = "no"
while answer != "yes":
    answer = input("Are we there yet? ")
print("Hooray! We are at the beach!")
```

OUTPUT

```
Are we there yet? no
Are we there yet? no
Are we there yet? yes
Hooray! We are at the beach!
```



for or while?

Both are loops. Pick the one that fits the job!

Use for when...

You know how many times to repeat.

You want to go through letters or a list.

```
for i in range(10):
```

Use while when...

You repeat until something happens.

You do not know how many turns.

```
while guess != "yes":
```

Quiz Time!

Q1 When does a while loop stop?

- A) After 10 turns
- B) When its question is False
- C) Never

Q2 How do you stop an infinite loop?

- A) Ctrl + C
- B) Press F5
- C) Type stop

Q3 `n = 3`, `while n > 0:`, `n -= 1`. How many turns?

- A) 2
- B) 3
- C) 4

Quiz Time! — Answers

Q1 **When does a while loop stop?**

A) After 10 turns

B) When its question is False

C) Never

Why? while keeps going as long as the question is True.

Q2 **How do you stop an infinite loop?**

A) Ctrl + C

B) Press F5

C) Type stop

Why? Ctrl + C stops a running program in the Shell.

Q3 `n = 3`, `while n > 0:`, `n -= 1`. **How many turns?**

A) 2

B) 3

C) 4

Why? n goes 3, 2, 1, then 0 makes it stop. 3 turns.

Module 14 Recap & Challenge

WHAT WE LEARNED

- `while` repeats while a question is True.
- Make the variable before the loop.
- Change something inside the loop!
- Ctrl + C stops a runaway program.
- Use while when you do not know how many turns.

CHALLENGE TIME!

Make a 'Times Table Tester'. Keep asking `What is 6 x 7?` until the player types 42. Then count how many guesses it took!



MODULE 15 OF 25

15 Loop Power-Ups

Time to give your loops super powers!

IN THIS MODULE YOU WILL:

- Escape a loop early with break
- Skip a turn with continue
- Print on one line with end
- Put loops inside loops



break: Escape the Loop!

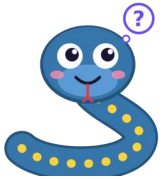
break jumps straight out of the loop, even if it is not finished.

main.py

```
for n in range(1, 11):
    if n == 5:
        break
    print(n)
print("Escaped!")
```

OUTPUT

```
1
2
3
4
Escaped!
```



PIP SAYS

When n becomes 5, break runs before the print, so 5 is never shown.

A Game Loop with break

while True loops forever, so we use break to get out.

main.py

```
while True:
    word = input("Say a word (or quit): ")
    if word == "quit":
        break
    print(word.upper() + "!!!")
print("Bye!")
```

OUTPUT

```
Say a word (or quit): hello
HELLO!!!
Say a word (or quit): pizza
PIZZA!!!
Say a word (or quit): quit
Bye!
```



PIP SAYS

Games often use while True with break. The game runs until the player quits!

continue: Skip a Turn

continue skips the rest of this turn and goes straight to the next one.

main.py

```
1 for n in range(1, 6):
2     if n == 3:
3         continue
4     print(n)
```

OUTPUT

```
1
2
4
5
```



PIP SAYS

3 is skipped, but the loop carries on with 4 and 5.

Skip the Odd Numbers

continue is useful for skipping things you do not want.

main.py

```
for n in range(1, 11):
    if n % 2 == 1:
        continue
    print(n, "is even")
```

OUTPUT

```
2 is even
4 is even
6 is even
8 is even
10 is even
```

Stay on One Line: end

Normally print() starts a new line. **end=" "** puts a space at the end instead.

main.py

```
1 for i in range(5):
2     print(i, end=" ")
3 print()
4 print("All done!")
```

OUTPUT

```
0 1 2 3 4
All done!
```



PIP SAYS

The empty print() on line 3 moves us onto a new line again.

Loops Inside Loops

A **nested loop** is a loop inside a loop. The inner loop runs ALL its turns for each outer turn.

1

Outer loop: row 0. Inner loop prints 4 stars.

2

Outer loop: row 1. Inner loop prints 4 stars.

3

Outer loop: row 2. Inner loop prints 4 stars.



PIP SAYS

It is like a clock! For every hour, the minutes go all the way round.

A Grid of Stars

3 rows with 4 stars each. That is 12 stars!

main.py

```
1 for row in range(3):
2     for col in range(4):
3         print("*", end=" ")
4     print()
```

OUTPUT

```
* * * *
* * * *
* * * *
```



PIP SAYS

The empty print() is in the OUTER loop, so it starts a new line after each row.

Star Triangle

Each row has one more star than the last.

main.py

```
1 for i in range(1, 6):
2     print("*" * i)
```

OUTPUT

```
*
**
***
****
*****
```



PIP SAYS

Try printing spaces first to make a pyramid!

Times Table Square

A nested loop makes a whole times table grid.

main.py

```
for a in range(1, 6):
    for b in range(1, 6):
        print(f"{a * b:4}", end="")
    print()
```

OUTPUT

```
   1   2   3   4   5
   2   4   6   8  10
   3   6   9  12  15
   4   8  12  16  20
   5  10  15  20  25
```



PIP SAYS

:4 makes every number take up 4 spaces, so the columns line up.

Quiz Time!

Q1 What does `break` do?

- A) Ends the loop
- B) Skips one turn
- C) Breaks Python

Q2 What does `continue` do?

- A) Ends the loop
- B) Skips to the next turn
- C) Repeats the turn

Q3 An outer loop runs 3 times and an inner loop runs 2 times. How many inner turns in total?

- A) 5
- B) 6
- C) 3

Quiz Time! — Answers

Q1 **What does `break` do?**

A) Ends the loop

B) Skips one turn

C) Breaks Python

Why? break jumps right out of the loop.

Q2 **What does `continue` do?**

A) Ends the loop

B) Skips to the next turn

C) Repeats the turn

Why? continue skips the rest of this turn and moves on.

Q3 **An outer loop runs 3 times and an inner loop runs 2 times. How many inner turns in total?**

A) 5

B) 6

C) 3

Why? The inner loop runs 2 times for each of the 3 outer turns: 3 x 2 = 6.

Module 15 Recap & Challenge

WHAT WE LEARNED

- `break` leaves a loop early.
- `continue` skips to the next turn.
- `while True` + `break` makes game loops.
- `end=" "` keeps printing on the same line.

- Nested loops are loops inside loops.

CHALLENGE TIME!

Draw a square of `#` symbols 5 wide and 5 tall using nested loops.
Bonus: make a hollow square!



MODULE 16 OF 25

16 Lists: Collections

What if you want to remember lots of things at once? Use a list!

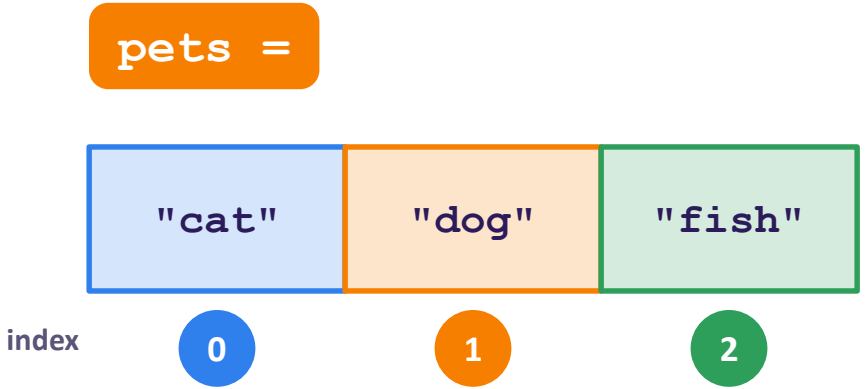
IN THIS MODULE YOU WILL:

- Keep lots of things in one list
- Get items using their index
- Loop through every item in a list



A List Holds Many Things

1. A **list** stores lots of values in one variable.
2. Use **square brackets** [ ] with **commas** between items.
3. The items stay in order, like children in a line.



Make Your First List

Print the whole list, then count its items with len().

main.py

```
1 pets = ["cat", "dog", "fish"]
2 print(pets)
3 print(len(pets))
```

OUTPUT

```
['cat', 'dog', 'fish']
3
```

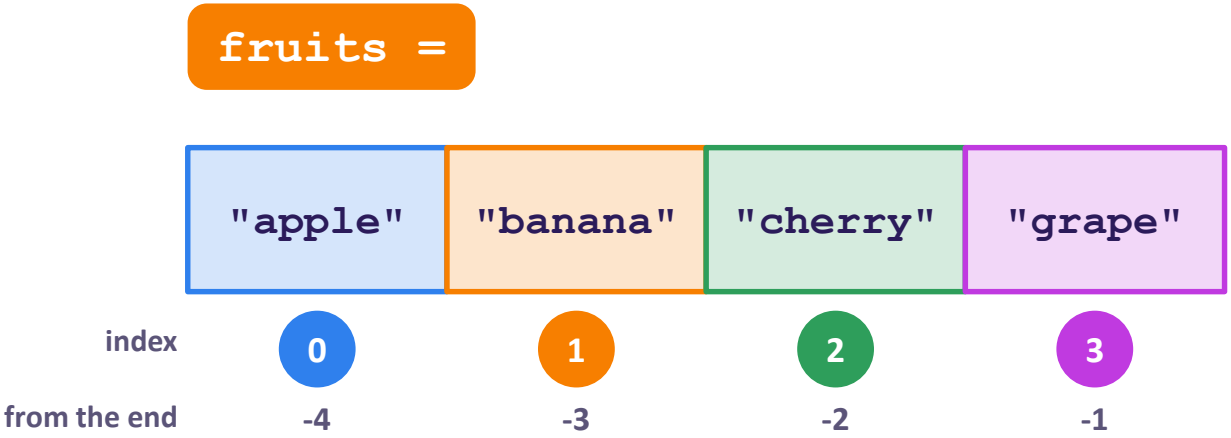


PIP SAYS

len() works on lists AND strings. It counts the items.

Every Item Has an Index

Lists count from **0**, just like strings do.





PIP SAYS

fruits[0] is the first item. fruits[-1] is the last item.

Getting Items Out

Put the index in square brackets after the list name.

main.py

```
1 fruits = ["apple", "banana", "cherry", "grape"]
2 print(fruits[0])
3 print(fruits[2])
4 print(fruits[-1])
```

OUTPUT

```
apple
cherry
grape
```

Changing an Item

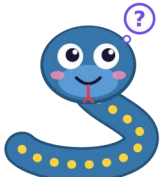
Lists can change! Store a new value at an index.

main.py

```
1 fruits = ["apple", "banana", "cherry"]
2 fruits[1] = "kiwi"
3 print(fruits)
```

OUTPUT

```
['apple', 'kiwi', 'cherry']
```



PIP SAYS

You cannot do this with strings, but you CAN with lists.

Loop Through a List

A for loop visits every item, one at a time.

main.py

```
1 fruits = ["apple", "banana", "cherry"]
2 for fruit in fruits:
3     print("I like", fruit)
```

OUTPUT

```
I like apple
I like banana
I like cherry
```



PIP SAYS

Use a name that makes sense. One fruit from fruits reads nicely!

Lists of Numbers

Python has handy tools for number lists.

main.py

```
scores = [7, 3, 9, 5]
print("Total:", sum(scores))
print("Best:", max(scores))
print("Worst:", min(scores))
print("Average:", sum(scores) / len(scores))
```

OUTPUT

```
Total: 24
Best: 9
Worst: 3
Average: 6.0
```



PIP SAYS

Average = total ÷ how many. 24 ÷ 4 = 6.0

Is It in the List?

in checks lists too!

main.py

```
1 pets = ["cat", "dog", "fish"]
2 print("dog" in pets)
3 print("lion" in pets)
4 if "cat" in pets:
5     print("Meow!")
```

OUTPUT

```
True
False
Meow!
```

Lists Can Hold Anything

A list can mix types. A list can even be empty!

main.py

```
1 stuff = ["Pip", 8, True, 3.5]
2 empty = []
3 print(stuff)
4 print(len(empty))
```

OUTPUT

```
['Pip', 8, True, 3.5]
0
```



PIP SAYS

An empty list [] is useful for filling up later. Next module!

Bug: Index Out of Range

This list has 3 items with index 0, 1 and 2. There is no index 3!

main.py

```
1 pets = ["cat", "dog", "fish"]
2 print(pets[3])
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(pets[3])
          ~~~~^^^
IndexError: list index out of range
```



PIP SAYS

IndexError means you asked for a place that does not exist. The last index is always len - 1.

Quiz Time!

Q1 What is `["a", "b", "c"][1]`?

A) "a"

B) "b"

C) "c"

Q2 What brackets make a list?

A) ( )

B) [ ]

C) < >

Q3 What does `len([4, 8, 15, 16])` give?

A) 4

B) 16

C) 43

Quiz Time! — Answers

Q1 **What is `["a", "b", "c"][1]`?**

A) "a"

B) "b"

C) "c"

Why? Index 1 is the second item, because lists start at 0.

Q2 **What brackets make a list?**

A) ( )

B) [ ]

C) < >

Why? Lists use square brackets.

Q3 **What does `len([4, 8, 15, 16])` give?**

A) 4

B) 16

C) 43

Why? There are 4 items in the list.

Module 16 Recap & Challenge

WHAT WE LEARNED

- A list stores many items: `[1, 2, 3]`.
- Indexes start at 0. `-1` is the last item.
- `list[1] = x` changes an item.
- `for item in list:` visits each item.
- `sum()`, `max()`, `min()` and `len()` work on lists.

CHALLENGE TIME!

Make a list of your 5 favourite foods. Print each one with a number in front, like `1. pizza`. (Hint: use a counter variable.)



MODULE 17 OF 25

17 More List Magic

Lists can grow, shrink and get sorted. Let's learn the tricks!

IN THIS MODULE YOU WILL:

- Add and remove items
- Sort lists and slice them
- Build lists with loops and input()



List Methods Cheat Sheet

Methods are tools that belong to the list. Use a dot to reach them.

Method	What it does	Example
`.append(x)`	Adds x to the end	`bag.append("pen")`
`.insert(i, x)`	Puts x at index i	`bag.insert(0, "map")`
`.remove(x)`	Removes the first x	`bag.remove("pen")`
`.pop()`	Takes off the last item	`bag.pop()`
`.sort()`	Sorts the list	`bag.sort()`
`.reverse()`	Flips the order	`bag.reverse()`

append: Add to the End

Start with an empty list and fill it up.

main.py

```
1 shopping = []
2 shopping.append("milk")
3 shopping.append("eggs")
4 shopping.append("bread")
5 print(shopping)
```

OUTPUT

```
['milk', 'eggs', 'bread']
```



insert, remove and pop

Put items anywhere, take them out by name, or pop off the last one.

main.py

```
1 team = ["Ali", "Zoe", "Sam"]
2 team.insert(0, "Mia")
3 print(team)
4 team.remove("Zoe")
5 print(team)
6 last = team.pop()
7 print(last, "has left")
8 print(team)
```

OUTPUT

```
['Mia', 'Ali', 'Zoe', 'Sam']
['Mia', 'Ali', 'Sam']
Sam has left
['Mia', 'Ali']
```



PIP SAYS

pop() gives back the item it removed, so you can store it.

Sorting Lists

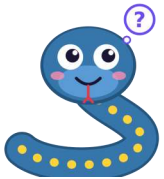
.sort() changes the list. **sorted()** makes a new sorted copy.

main.py

```
1 numbers = [5, 2, 9, 1, 7]
2 numbers.sort()
3 print(numbers)
4 numbers.reverse()
5 print(numbers)
6 names = ["Zoe", "Ali", "Mia"]
7 print(sorted(names))
8 print(names)
```

OUTPUT

```
[1, 2, 5, 7, 9]
[9, 7, 5, 2, 1]
['Ali', 'Mia', 'Zoe']
['Zoe', 'Ali', 'Mia']
```



PIP SAYS

Look at the last line: sorted() did not change the original names list.

Slicing Lists

Slices work on lists exactly like on strings.

main.py

```
1 planets = ["Mercury", "Venus", "Earth", "Mars"]
2 print(planets[0:2])
3 print(planets[2:])
4 print(planets[-2:])
```

OUTPUT

```
['Mercury', 'Venus']
['Earth', 'Mars']
['Earth', 'Mars']
```



PIP SAYS

planets[-2:] means: the last two items.

Counting and Finding

.count() says how many times something appears. **.index()** says where it first appears.

main.py

```
1 letters = ["a", "b", "a", "c", "a"]
2 print(letters.count("a"))
3 print(letters.index("c"))
```

OUTPUT

```
3
3
```

Build a List with a Loop

Use a loop to fill a list automatically.

main.py

```
squares = []
for n in range(1, 6):
    squares.append(n * n)
print(squares)
```

OUTPUT

```
[1, 4, 9, 16, 25]
```



PIP SAYS

These are square numbers: 1 x 1, 2 x 2, 3 x 3...

Lists and Strings: split and join

.split() chops a string into a list. **.join()** glues a list back into a string.

main.py

```
1 colours = "red green blue".split()
2 print(colours)
3 print(len(colours))
4 print(" and ".join(colours))
```

▶ OUTPUT

```
['red', 'green', 'blue']
3
red and green and blue
```



PIP SAYS

Whatever is before .join is the glue that goes between the items.

Party Guest List

Collect answers into a list until the user types **done**.

party.py

```
guests = []
while True:
    name = input("Name? (or done) ")
    if name == "done":
        break
    guests.append(name)
print(f"You invited {len(guests)} friends:")
for g in guests:
    print("-", g)
```

OUTPUT

```
Name? (or done) Ali
Name? (or done) Zoe
Name? (or done) Sam
Name? (or done) done
You invited 3 friends:
- Ali
- Zoe
- Sam
```

Quiz Time!

Q1 Which method adds an item to the END?

A) `.insert()`

B) `.append()`

C) `.pop()`

Q2 `x = [3, 1, 2]` then `x.sort()`. What is x?

A) [3, 1, 2]

B) [1, 2, 3]

C) [3, 2, 1]

Q3 What does `"a b c".split()` give?

A) `["a", "b", "c"]`

B) `"abc"`

C) `3`

Quiz Time! — Answers

Q1 **Which method adds an item to the END?**

A) `.insert()`

B) `.append()`

C) `.pop()`

Why? append adds to the end of the list.

Q2 `x = [3, 1, 2]` **then** `x.sort()`**. What is x?**

A) [3, 1, 2]

B) [1, 2, 3]

C) [3, 2, 1]

Why? sort puts the numbers in order, smallest first.

Q3 **What does** `"a b c".split()` **give?**

A) `["a", "b", "c"]`

B) `"abc"`

C) `3`

Why? split breaks a string into a list of words.

Module 17 Recap & Challenge

WHAT WE LEARNED

- `.append()` adds to the end. `.insert()` adds anywhere.
- `.remove()` takes out by value. `.pop()` takes the last.
- `.sort()` changes the list. `sorted()` makes a copy.
- Slices work on lists.
- `.split()` and `.join()` connect strings and lists.

CHALLENGE TIME!

Make a 'Top 3' program. Ask for 5 numbers, put them in a list, sort it, and print the 3 biggest. (Hint: a slice like `[-3:]`.)



MODULE 18 OF 25

18 Tuples and Sets

Lists have two cousins. Let's meet them!

IN THIS MODULE YOU WILL:

- Use tuples for things that never change
- Unpack tuples into variables
- Use sets to remove repeats



Tuples: Locked Lists

1. A **tuple** is like a list, but it **cannot change**.
2. Tuples use **round brackets** ( ).
3. Use them for things that stay the same, like a birthday or a map position.

```
(14, "March")
```

A birthday never changes, so a tuple is perfect.



PIP SAYS

Say it like **too-pull** or **tup-ul**. Both are fine!

Using a Tuple

Get items from a tuple with an index, just like a list.

main.py

```
1 birthday = (14, "March")
2 print(birthday)
3 print(birthday[0])
4 print(birthday[1])
5 print(len(birthday))
```

OUTPUT

```
(14, 'March')
14
March
2
```

Bug: Tuples Are Locked!

Trying to change a tuple causes an error.

main.py

```
1 birthday = (14, "March")
2 birthday[0] = 15
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    birthday[0] = 15
    ~~~~~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```



PIP SAYS

does not support item assignment means: you are not allowed to change this.

Unpacking a Tuple

Put each item of a tuple into its own variable in one step!

main.py

```
1 position = (3, 5)
2 x, y = position
3 print("x is", x)
4 print("y is", y)
```

OUTPUT

```
x is 3
y is 5
```

The Magic Swap

Unpacking lets you swap two variables in one line.

main.py

```
1 a = "cat"
2 b = "dog"
3 a, b = b, a
4 print(a)
5 print(b)
```

OUTPUT

```
dog
cat
```



PIP SAYS

Most languages need 3 lines to do this. Python needs just one!

Sets: No Repeats Allowed!

1. A **set** keeps only **one of each** thing.
2. Sets use **curly brackets** { }.
3. Sets have **no order**, so they have no index numbers.

You put in
"red"
"blue"
"red"
"green"
"blue"

The set keeps
"red"
"blue"
"green"

Repeats Disappear

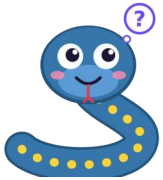
We put in 5 colours, but only 3 are different.

main.py

```
1 colours = {"red", "blue", "red", "green", "blue"}
2 print(len(colours))
3 print(sorted(colours))
```

OUTPUT

```
3
['blue', 'green', 'red']
```



PIP SAYS

We use sorted() to print the set in a neat order. A set on its own can print in any order.

Find the Different Names

Turn a list into a set to throw away the repeats.

main.py

```
1 names = ["Ali", "Zoe", "Ali", "Sam", "Zoe"]
2 unique = set(names)
3 print(len(unique), "different names")
4 print(sorted(unique))
```

OUTPUT

```
3 different names
['Ali', 'Sam', 'Zoe']
```



Sticker Collection

Make an empty set with **set()**, then **.add()** things.

main.py

```
1 stickers = set()
2 stickers.add("star")
3 stickers.add("heart")
4 stickers.add("star")
5 print(len(stickers))
6 print("star" in stickers)
7 print("moon" in stickers)
```

OUTPUT

```
2
True
False
```

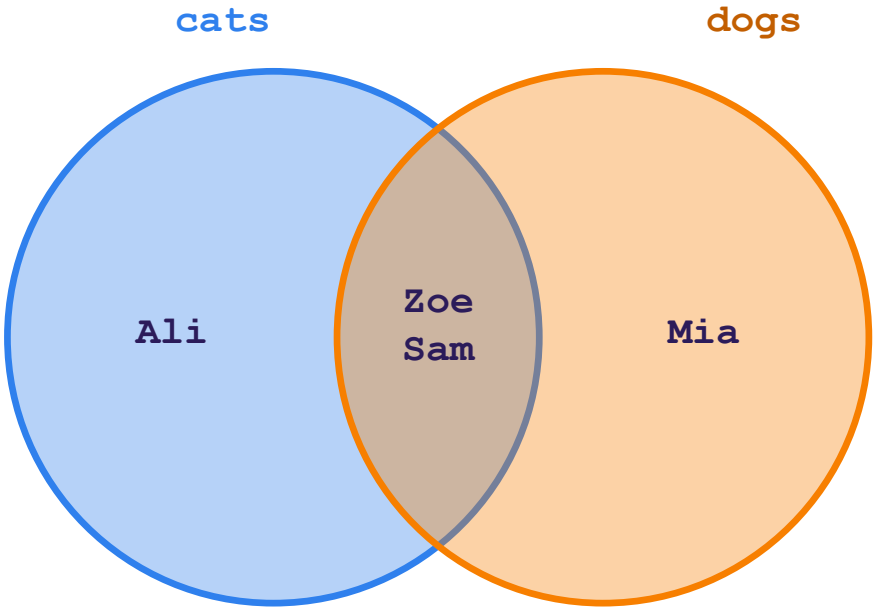


PIP SAYS

We added star twice, but it only counts once!

Sets Can Be Compared

Who likes cats? Who likes dogs? Who likes **both**?





PIP SAYS

The middle part is called the **intersection**. Everyone together is called the **union**.

Both and Either: & and |

& finds who is in BOTH sets. **|** puts both sets together.

main.py

```
1 cats = {"Ali", "Zoe", "Sam"}
2 dogs = {"Zoe", "Sam", "Mia"}
3 print("Both:", sorted(cats & dogs))
4 print("Either:", sorted(cats | dogs))
```

OUTPUT

```
Both: ['Sam', 'Zoe']
Either: ['Ali', 'Mia', 'Sam', 'Zoe']
```



PIP SAYS

This matches the picture on the last slide!

List, Tuple or Set?

Pick the right container for the job!

	List	Tuple	Set
Brackets	[1, 2]	(1, 2)	{1, 2}
Can change?	Yes	No	Yes
Keeps order?	Yes	Yes	No
Allows repeats?	Yes	Yes	No

Quiz Time!

Q1 Which can NOT be changed?

- A) A list
- B) A tuple
- C) A set

Q2 How many items are in `{"a", "a", "b"}`?

- A) 3
- B) 2
- C) 1

Q3 What brackets does a tuple use?

- A) ( )
- B) [ ]
- C) { }

Quiz Time! — Answers

Q1 **Which can NOT be changed?**

A) A list

B) A tuple

C) A set

Why? Tuples are locked once you make them.

Q2 **How many items are in `{"a", "a", "b"}`?**

A) 3

B) 2

C) 1

Why? Sets throw away repeats, leaving a and b.

Q3 **What brackets does a tuple use?**

A) ( )

B) [ ]

C) { }

Why? Tuples use round brackets.

Module 18 Recap & Challenge

WHAT WE LEARNED

- Tuples use `( )` and cannot change.
- `x, y = (3, 5)` unpacks a tuple.
- Sets use `{ }` and remove repeats.
- `set()` makes an empty set. `.add()` adds to it.
- `&` finds both, `|` joins sets.

CHALLENGE TIME!

Ask 3 friends to name their favourite animals (store them in a list). Then use a set to print how many DIFFERENT animals were chosen.



MODULE 19 OF 25

19 Dictionaries

A dictionary lets you look things up by name.

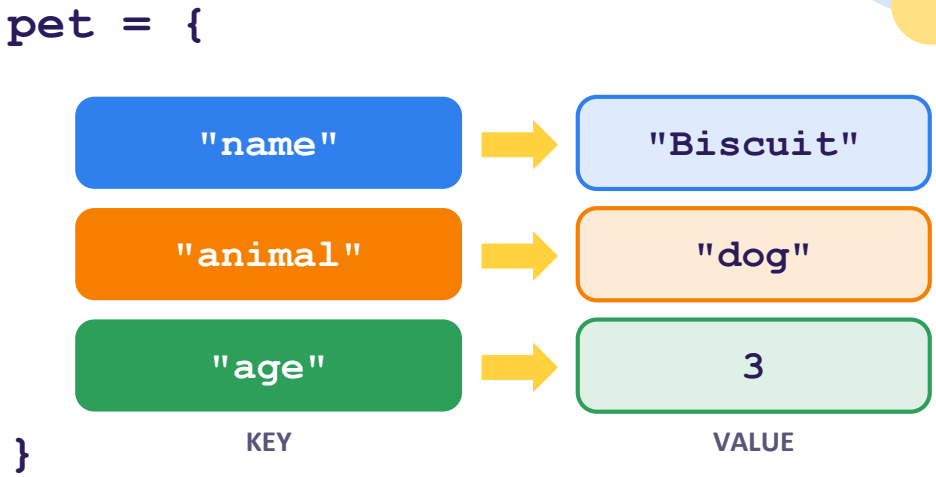
IN THIS MODULE YOU WILL:

- Store pairs of keys and values
- Look up, add and change values
- Loop through a dictionary



Look It Up!

1. A real dictionary pairs a **word** with its **meaning**.
2. A Python **dictionary** pairs a **key** with a **value**.
3. You find a value by asking for its key.



How to Write a Dictionary

You can write a dictionary on many lines to make it easy to read.

main.py

```
pet = {
    "name": "Biscuit",
    "animal": "dog",
    "age": 3,
}
```

Curly brackets { } start and end it.

"name" is a key. Keys are often strings.

A **colon :** joins each key to its value.

Values can be any type. A **comma** goes after each pair.

Looking Up Values

Use the key in square brackets to get its value.

main.py

```
pet = {"name": "Biscuit", "animal": "dog", "age": 3}
print(pet["name"])
print(pet["age"])
print(f"{pet['name']} is a {pet['animal']}.")
```

OUTPUT

```
Biscuit
3
Biscuit is a dog.
```



PIP SAYS

Inside an f-string with double quotes, use single quotes for the key.

Adding and Changing

Give an old key a new value to change it. Use a new key to add a pair.

main.py

```
1 pet = {"name": "Biscuit", "age": 3}
2 pet["age"] = 4
3 pet["toy"] = "ball"
4 print(pet)
5 print(len(pet))
```

OUTPUT

```
{'name': 'Biscuit', 'age': 4, 'toy': 'ball'}
3
```



PIP SAYS

Each key can only appear once in a dictionary.

Looping Through a Dictionary

.items() gives you each key AND value together.

main.py

```
1 ages = {"Ali": 8, "Zoe": 9, "Sam": 7}
2 for name, age in ages.items():
3     print(f"{name} is {age}")
```

OUTPUT

```
Ali is 8
Zoe is 9
Sam is 7
```

Bug: Missing Key!

There is no key called "colour" in this dictionary.

main.py

```
1 pet = {"name": "Biscuit"}
2 print(pet["colour"])
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(pet["colour"])
          ~~~^^^^^^^^^^
KeyError: 'colour'
```



PIP SAYS

KeyError means Python looked for a key that is not there.

Safe Look-Ups with .get()

.get() does not crash. It gives None, or a backup value you choose.

main.py

```
1 pet = {"name": "Biscuit"}
2 print(pet.get("colour"))
3 print(pet.get("colour", "unknown"))
4 print("name" in pet)
```

OUTPUT

```
None
unknown
True
```



PIP SAYS

in checks if a KEY is in the dictionary.

Pirate Translator

Swap words using a dictionary. Arrr!

main.py

```
pirate = {"hello": "ahoy", "friend": "matey", "yes": "aye"}
sentence = "hello friend"
for word in sentence.split():
    print(pirate.get(word, word), end=" ")
print()
```

OUTPUT

```
ahoy matey
```



PIP SAYS

If a word is not in the dictionary, .get(word, word) keeps the original word.

Game Scoreboard

Dictionaries are perfect for keeping score.

main.py

```
1 scores = {"Ali": 0, "Zoe": 0}
2 scores["Ali"] += 10
3 scores["Zoe"] += 15
4 scores["Ali"] += 20
5 for player, points in scores.items():
6     print(player, "has", points, "points")
```

OUTPUT

```
Ali has 30 points
Zoe has 15 points
```

Count the Letters

Use a dictionary to count how many times each letter appears.

main.py

```
word = "banana"
counts = {}
for letter in word:
    counts[letter] = counts.get(letter, 0) + 1
print(counts)
```

OUTPUT

```
{'b': 1, 'a': 3, 'n': 2}
```



PIP SAYS

counts.get(letter, 0) gives 0 the first time we see a letter.

Quiz Time!

Q1 `d = {"a": 1, "b": 2}`. What is `d["b"]`?

- A) 1
- B) 2
- C) "b"

Q2 What error happens when a key is missing?

- A) KeyError
- B) NameError
- C) IndexError

Q3 Which brackets make a dictionary?

- A) [ ]
- B) ( )
- C) { }

Quiz Time! — Answers

Q1 `d = {"a": 1, "b": 2}`. What is `d["b"]`?

A) 1

B) 2

C) "b"

Why? The key "b" is paired with the value 2.

Q2 What error happens when a key is missing?

A) KeyError

B) NameError

C) IndexError

Why? A missing key gives a KeyError.

Q3 Which brackets make a dictionary?

A) [ ]

B) ( )

C) { }

Why? Dictionaries use curly brackets, with key: value pairs.

Module 19 Recap & Challenge

WHAT WE LEARNED

- Dictionaries store `key: value` pairs.
- `d["key"]` looks up a value.
- `d["key"] = value` adds or changes.
- `.get()` looks up safely.
- `for k, v in d.items():` loops through pairs.

CHALLENGE TIME!

Make a dictionary of 3 friends and their favourite colours. Ask the user for a name and print that friend's colour, or `I don't know them!`



MODULE 20 OF 25

20 Functions: Your Own Commands

You have used print() and len(). Now make your own!

IN THIS MODULE YOU WILL:

- Make your own commands with def
- Call a function as many times as you like
- Send information in with parameters



What Is a Function?

1. A **function** is a named set of instructions.
2. You write it **once**, then use it **again and again**.
3. print(), input() and len() are functions that come with Python.



Something goes in, the function does its job.

How to Make a Function

Making a function is called **defining** it.

main.py

```
1 def say_hello():
2     print("Hello!")
3     print("Nice to see you!")
4
5 say_hello()
```

def means define: make a new function.

Then the **name**, brackets **( )** and a **colon**.

The indented lines are the **body**: what the function does.

Calling the function makes it run.

Define Once, Use Twice!

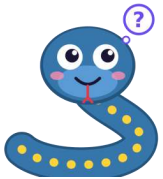
Each call runs the whole body again.

main.py

```
1 def say_hello():
2     print("Hello!")
3     print("Nice to see you!")
4
5 say_hello()
6 say_hello()
```

OUTPUT

```
Hello!
Nice to see you!
Hello!
Nice to see you!
```



PIP SAYS

Defining a function does not run it. Only calling it does!

Why Use Functions?

Big programs are built from lots of small functions.



Reuse

Write code once and use it many times.



Tidy

Break a big program into small, named pieces.



Easy to fix

Fix a bug in one place and it is fixed everywhere.



Share

Other coders can use your functions too!

Bug: Call Before Define

Python reads from top to bottom. On line 1, cheer() does not exist yet!

main.py

```
1 cheer()
2
3 def cheer():
4     print("Hip hip hooray!")
```

OUTPUT

```
Traceback (most recent call last):
  File "main.py", line 1, in <module>
    cheer()
    ^^^^^
NameError: name 'cheer' is not defined
```



PIP SAYS

Always define your functions at the TOP of your program.

A Song with a Chorus

Songs repeat their chorus. Functions are perfect for that!

main.py

```
1 def chorus():
2     print("Coding is fun, fun, fun!")
3     print("Let's code for everyone!")
4
5 print("Verse 1: I wrote a loop today.")
6 chorus()
7 print("Verse 2: My bugs all ran away.")
8 chorus()
```

OUTPUT

```
Verse 1: I wrote a loop today.
Coding is fun, fun, fun!
Let's code for everyone!
Verse 2: My bugs all ran away.
Coding is fun, fun, fun!
Let's code for everyone!
```

Functions Calling Functions

One function can call another one.

main.py

```
1 def line():
2     print("~" * 16)
3
4 def banner():
5     line()
6     print("   PIP'S PROGRAM")
7     line()
8
9 banner()
```

OUTPUT

```
~~~~~~~~~~~~~~~~
   PIP'S PROGRAM
~~~~~~~~~~~~~~~~
```



PIP SAYS

banner() calls line() twice. Functions can be building blocks!

Parameters: Sending Things In

MODULE 20

Parameters let you give a function some information.

main.py

```
1 def greet(name):
2     print(f"Hello, {name}!")
3
4 greet("Mia")
```

name is a parameter: a box that gets filled when we call.

Use the parameter inside the body.

"Mia" is the value we send in. It goes into name.

One Function, Many Friends

Each call can send in a different value.

main.py

```
1 def greet(name):
2     print(f"Hello, {name}!")
3
4 greet("Mia")
5 greet("Ali")
6 greet("Zoe")
```

OUTPUT

```
Hello, Mia!
Hello, Ali!
Hello, Zoe!
```

Two Parameters

Separate parameters with commas. They fill up in order.

main.py

```
1 def describe_pet(name, animal):
2     print(f"{name} is a {animal}.")
3
4 describe_pet("Biscuit", "dog")
5 describe_pet("Nibbles", "hamster")
```

OUTPUT

```
Biscuit is a dog.
Nibbles is a hamster.
```



PIP SAYS

The first value goes in name, the second goes in animal.

Quiz Time!

Q1 Which word starts a function?

A) `fun`

B) `def`

C) `make`

Q2 What makes a function run?

A) Defining it

B) Calling it

C) Saving the file

Q3 In `def greet(name):`, what is name?

A) A parameter

B) A loop

C) A string

Quiz Time! — Answers

Q1 **Which word starts a function?**

A) `fun`

B) `def`

C) `make`

Why? def is short for define.

Q2 **What makes a function run?**

A) Defining it

B) Calling it

C) Saving the file

Why? A function only runs when you call it, like say_hello().

Q3 **In `def greet(name):`, what is name?**

A) A parameter

B) A loop

C) A string

Why? name is a parameter. It is filled when you call greet.

Module 20 Recap & Challenge

WHAT WE LEARNED

- `def name():` defines a function.
- Calling `name()` runs it.
- Define functions before you call them.
- Parameters send information in: `def greet(name):`
- Functions can call other functions.

CHALLENGE TIME!

Make a function `draw_box(width)` that prints a box of `#` that wide and 3 rows tall. Call it with 3 different widths!



MODULE 21 OF 25

21 Functions That Give Back

Some functions do a job. Others work out an answer and hand it back!

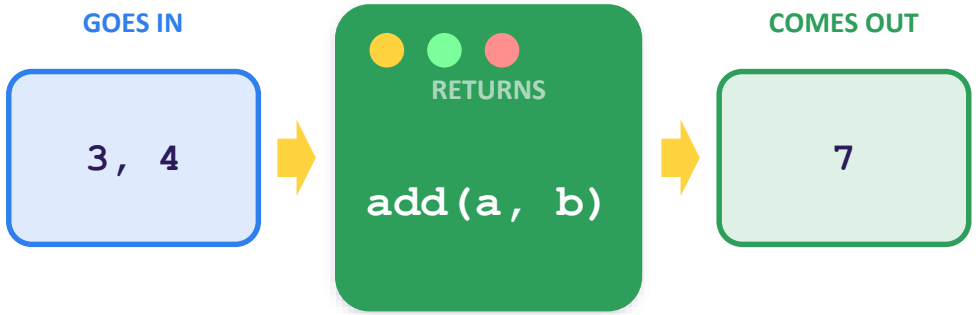
IN THIS MODULE YOU WILL:

- Send answers back with return
- Know the difference between print and return
- Give parameters default values



return Hands Back an Answer

1. **return** sends a value back to the code that called the function.
2. You can store the answer in a variable, or use it straight away.
3. len() works like this: it **returns** a number.



add(3, 4) gives back 7.

Inside a Function That Returns

The returned value comes out where the function was called.

main.py

```
1 def double(n):
2     return n * 2
3
4 answer = double(5)
5 print(answer)
```

n is the parameter. It holds the number we send in.

return works out n * 2 and sends it back.

The answer, **10**, is caught in a variable.

Now we can print it, or use it again.

Using Returned Values

A returned value can go straight into print(), or even into another call!

main.py

```
1 def double(n):
2     return n * 2
3
4 print(double(5))
5 print(double(21))
6 big = double(double(3))
7 print(big)
```

OUTPUT

```
10
42
12
```



PIP SAYS

double(double(3)) works from the inside out: double(3) is 6, and double(6) is 12.

print or return?

These two are easy to mix up!

print()

Shows something on the screen.

For people to read.

Your code cannot use it afterwards.

return

Gives a value back to your code.

For the program to use.

Store it, add to it, print it later.



PIP SAYS

Think: print is like saying the answer out loud. return is like handing over a note with the answer.

Area of a Rectangle

Because the answers come back as numbers, we can add them together.

main.py

```
1 def area(width, height):
2     return width * height
3
4 room = area(4, 3)
5 garden = area(10, 6)
6 print("Room:", room)
7 print("Garden:", garden)
8 print("Total:", room + garden)
```

OUTPUT

```
Room: 12
Garden: 60
Total: 72
```

Returning True or False

A function can return a Boolean, ready to use in an if.

main.py

```
def is_even(n):
    return n % 2 == 0

for n in [4, 7, 10]:
    if is_even(n):
        print(n, "is even")
    else:
        print(n, "is odd")
```

OUTPUT

```
4 is even
7 is odd
10 is even
```

Returning a String

Functions can return words too.

main.py

```
1 def pirate_name(name):
2     return "Captain " + name + " Bluebeard"
3
4 print(pirate_name("Zoe"))
5 print(pirate_name("Ali"))
```

OUTPUT

```
Captain Zoe Bluebeard
Captain Ali Bluebeard
```



Default Values

Give a parameter a **default**. It is used when you do not send a value.

main.py

```
1 def greet(name, greeting="Hello"):
2     return f"{greeting}, {name}!"
3
4 print(greet("Mia"))
5 print(greet("Mia", "Good morning"))
```

OUTPUT

```
Hello, Mia!
Good morning, Mia!
```



PIP SAYS

Parameters with defaults must come after ones without them.

return Stops the Function

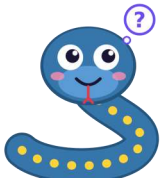
As soon as Python reaches return, it leaves the function.

main.py

```
1 def check_age(age):
2     if age < 0:
3         return "That is not a real age!"
4     return f"You are {age}."
5
6 print(check_age(-3))
7 print(check_age(9))
```

OUTPUT

```
That is not a real age!
You are 9.
```



PIP SAYS

For -3, the first return runs, so the last line of the function is never reached.

Variables Stay Inside

A variable made inside a function only lives inside that function.

main.py

```
1 def make_snack():
2     snack = "apple"
3     return snack
4
5 print(make_snack())
6 print(snack)
```

OUTPUT

```
apple
Traceback (most recent call last):
  File "main.py", line 6, in <module>
    print(snack)
          ^^^^^
NameError: name 'snack' is not defined
```



PIP SAYS

This is called **local scope**. To get the value out, use return!

Temperature Converter

A useful function to change Celsius into Fahrenheit.

main.py

```
1 def c_to_f(celsius):
2     return celsius * 9 / 5 + 32
3
4 for c in [0, 20, 37, 100]:
5     print(f"{c}°C is {c_to_f(c)}°F")
```

OUTPUT

```
0°C is 32.0°F
20°C is 68.0°F
37°C is 98.6°F
100°C is 212.0°F
```



PIP SAYS

0°C is freezing and 100°C is boiling. Water knows maths!

Quiz Time!

Q1 **What does return do?**

A) Prints a value

B) Sends a value back

C) Ends Python

Q2 `def f(x): return x + 1`. **What is `f(4)`?**

A) 4

B) 5

C) x + 1

Q3 `def hi(name="Pip")`. **What is name if you call `hi()`?**

A) `"Pip"`

B) Nothing

C) An error

Quiz Time! — Answers

Q1 **What does return do?**

A) Prints a value

B) Sends a value back

C) Ends Python

Why? return sends a value back to where the function was called.

Q2 `def f(x): return x + 1`. **What is `f(4)`?**

A) 4

B) 5

C) x + 1

Why? x is 4, so x + 1 is 5.

Q3 `def hi(name="Pip")`. **What is name if you call `hi()`?**

A) `"Pip"`

B) Nothing

C) An error

Why? The default value is used when nothing is sent.

Module 21 Recap & Challenge

WHAT WE LEARNED

- `return` sends a value back.
- print shows; return gives.
- Returned values can be stored and used again.
- Default values: `def greet(name, greeting="Hi"):`
- Variables inside a function stay inside.

CHALLENGE TIME!

Write `biggest(a, b)` that returns the bigger of two numbers. Then write `pounds_to_pence(p)`. Test them with print()!



MODULE 22 OF 25

22 Modules and Random

Python comes with toolboxes full of extra commands. They are called modules.

IN THIS MODULE YOU WILL:

- Import extra tools from modules
- Roll dice and pick random things
- Use the math and time modules



Modules Are Toolboxes

To use a module, you **import** it at the top of your program.

random

Random numbers and choices. Great for games!

math

Square roots, pi and more maths tools.

time

Pause your program and work with time.

turtle

Draw pictures. Coming next module!



PIP SAYS

These modules all come free with Python. You do not need to download anything!

Roll a Dice!

random.randint(1, 6) picks a whole number from 1 to 6. Both ends are included!

main.py

```
1 import random
2
3 roll = random.randint(1, 6)
4 print("You rolled a", roll)
```

OUTPUT

```
You rolled a 2
```



PIP SAYS

Random numbers change every run, so yours may be different!

Roll Lots of Dice

Every call gives a new random number.

main.py

```
1 import random
2
3 for i in range(5):
4     print(random.randint(1, 6), end=" ")
5 print()
```

OUTPUT

```
4 5 4 4 5
```



PIP SAYS

Random numbers change every run, so yours may be different!

Pick a Random Thing

random.choice() picks one item from a list.

main.py

```
1 import random
2
3 snacks = ["apple", "crisps", "yoghurt", "grapes"]
4 print("Today's snack is", random.choice(snacks))
```

OUTPUT

```
Today's snack is crisps
```



PIP SAYS

Random numbers change every run, so yours may be different!

Shuffle a List

random.shuffle() mixes up a list, like shuffling playing cards.

main.py

```
1 import random
2
3 cards = ["Ace", "King", "Queen", "Jack"]
4 random.shuffle(cards)
5 print(cards)
```

OUTPUT

```
['Ace', 'King', 'Jack', 'Queen']
```



PIP SAYS

Random numbers change every run, so yours may be different!

The math Module

Square roots, pi, and rounding down or up.

main.py

```
1 import math
2
3 print(math.sqrt(16))
4 print(math.pi)
5 print(math.floor(4.8))
6 print(math.ceil(4.2))
```

OUTPUT

```
4.0
3.141592653589793
4
5
```



PIP SAYS

floor always rounds DOWN. **ceil** (ceiling) always rounds UP.

Import Just What You Need

from ... import ... lets you skip the module name each time.

main.py

```
1 from random import randint
2 from math import sqrt
3
4 print(randint(1, 10))
5 print(sqrt(81))
```

OUTPUT

```
1
9.0
```



PIP SAYS

Random numbers change every run, so yours may be different!

A Slow Countdown with time

time.sleep(1) pauses for 1 second. Great for suspense!

main.py

```
1 import time
2
3 for n in range(3, 0, -1):
4     print(n)
5     time.sleep(1)
6 print("Go!")
```

OUTPUT

```
3
2
1
Go!
```



PIP SAYS

The output looks the same, but each number appears 1 second apart.

Magic 8-Ball

We do not even need to store the question!

main.py

```
1 import random
2
3 answers = ["Yes!", "No way!", "Maybe...", "Ask again later"]
4 input("Ask me a yes or no question: ")
5 print("The 8-ball says:", random.choice(answers))
```

OUTPUT

```
Ask me a yes or no question: Will it snow tomorrow?
The 8-ball says: No way!
```



PIP SAYS

Random numbers change every run, so yours may be different!

Flip a Coin 10 Times

Count how many heads come up.

main.py

```
import random

heads = 0
for i in range(10):
    if random.choice(["H", "T"]) == "H":
        heads += 1
print("Heads:", heads)
print("Tails:", 10 - heads)
```

OUTPUT

```
Heads: 5
Tails: 5
```



PIP SAYS

Random numbers change every run, so yours may be different!

Quiz Time!

Q1 What word loads a module?

- A) `load`
- B) `import`
- C) `get`

Q2 What can `random.randint(1, 3)` give?

- A) 1 or 2
- B) 1, 2 or 3
- C) Only 3

Q3 What does `math.sqrt(25)` give?

- A) 5.0
- B) 625
- C) 12.5

Quiz Time! — Answers

Q1 **What word loads a module?**

A) `load`

B) `import`

C) `get`

Why? import loads a module so you can use its tools.

Q2 **What can `random.randint(1, 3)` give?**

A) 1 or 2

B) 1, 2 or 3

C) Only 3

Why? randint includes both end numbers.

Q3 **What does `math.sqrt(25)` give?**

A) 5.0

B) 625

C) 12.5

Why? The square root of 25 is 5. math.sqrt gives a float.

Module 22 Recap & Challenge

WHAT WE LEARNED

- `import random` loads a module.
- `randint(a, b)` gives a random whole number.
- `choice()` and `shuffle()` work on lists.
- `math` has sqrt, pi, floor and ceil.
- `time.sleep()` pauses.

CHALLENGE TIME!

Make a 'Random Name Generator'. Use two lists (like `["Super", "Mega"]` and `["Cat", "Rocket"]`) and random.choice() to make 5 silly names!



MODULE 23 OF 25

23 Turtle Drawing

Python has a little turtle that draws wherever it walks!



IN THIS MODULE YOU WILL:

- Move a turtle to draw lines
- Draw squares, triangles and stars
- Use loops and colours to make art

Meet the Turtle

1. The **turtle** module opens a window with a turtle in the middle.
2. It starts facing **right**, with its pen **down**.
3. When it moves, it draws a line behind it!

Command	What it does
`forward(100)`	Move forward 100 steps
`backward(50)`	Move back 50 steps
`left(90)`	Turn left 90 degrees
`right(90)`	Turn right 90 degrees
`penup()`	Lift the pen: no line
`pendown()`	Put the pen back down

Your First Line

Make a turtle called **t**, then tell it to move.

main.py

```
1 import turtle
2
3 t = turtle.Turtle()
4 t.forward(150)
5
6 turtle.done()
```



PIP SAYS

turtle.done() keeps the window open so you can see your drawing.

TURTLE WINDOW

Turning Corners

Turtles turn in **degrees**. A whole turn is 360 degrees.

90 degrees

A quarter turn. Like a corner of a square.

180 degrees

A half turn. Now you face the other way.

360 degrees

A full turn. Back where you started!



PIP SAYS

To draw a shape with equal sides, turn **360 ÷ number of sides** at every corner.

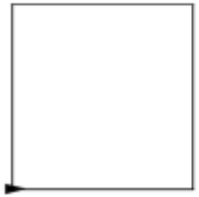
Draw a Square

4 sides and 4 turns of 90 degrees. A loop makes it short!

main.py

```
1 import turtle
2
3 t = turtle.Turtle()
4 for i in range(4):
5     t.forward(150)
6     t.left(90)
7
8 turtle.done()
```

TURTLE WINDOW





PIP SAYS

360 ÷ 4 = 90. That is why we turn 90!

Draw a Triangle

color() changes the pen colour. **pensize()** makes the line thicker.

main.py

```
import turtle

t = turtle.Turtle()
t.color("red")
t.pensize(4)
for i in range(3):
    t.forward(160)
    t.left(120)

turtle.done()
```

TURTLE WINDOW





PIP SAYS

360 ÷ 3 = 120. So a triangle turns 120 degrees at each corner.

A Shining Star

Turn 144 degrees five times and a star appears!

main.py

```
1 import turtle
2
3 t = turtle.Turtle()
4 t.color("gold")
5 t.pensize(3)
6 for i in range(5):
7     t.forward(200)
8     t.right(144)
9
10 turtle.done()
```

TURTLE WINDOW





PIP SAYS

Try other numbers instead of 144. What shapes can you find?

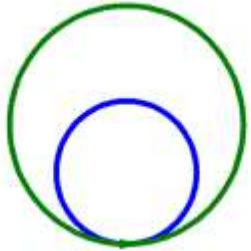
Circles

circle(60) draws a circle with a radius of 60 steps.

main.py

```
1  import turtle
2
3  t = turtle.Turtle()
4  t.pensize(3)
5  t.color("blue")
6  t.circle(60)
7  t.color("green")
8  t.circle(100)
9
10 turtle.done()
```

TURTLE WINDOW





PIP SAYS

The circle is drawn to the turtle's LEFT side.

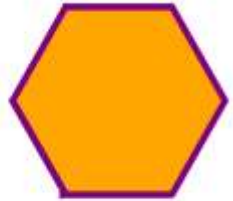
Filling In Colour

Draw between **begin_fill()** and **end_fill()** to colour the shape in.

main.py

```
1 import turtle
2
3 t = turtle.Turtle()
4 t.color("purple", "orange")
5 t.pensize(4)
6 t.begin_fill()
7 for i in range(6):
8     t.forward(100)
9     t.left(60)
10 t.end_fill()
11
12 turtle.done()
```

TURTLE WINDOW





PIP SAYS

color("purple", "orange") means a purple outline with an orange fill.

Pen Up, Pen Down

Lift the pen to move without drawing. This makes a dotted line!

main.py

```
1 import turtle
2
3 t = turtle.Turtle()
4 t.pensize(3)
5 for i in range(4):
6     t.pendown()
7     t.forward(40)
8     t.penup()
9     t.forward(30)
10
11 turtle.done()
```

TURTLE WINDOW



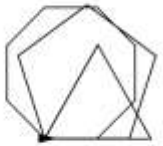
A Shape Function

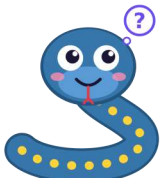
Combine functions and turtle to draw any shape!

main.py

```
1  import turtle
2
3  t = turtle.Turtle()
4
5  def polygon(sides, length):
6      for i in range(sides):
7          t.forward(length)
8          t.left(360 / sides)
9
10 polygon(3, 100)
11 polygon(5, 80)
12 polygon(8, 50)
13
14 turtle.done()
```

TURTLE WINDOW





PIP SAYS

One function draws triangles, pentagons and octagons!

Rainbow Spiral

Loops + lists + colours = amazing art!

main.py

```
import turtle

t = turtle.Turtle()
colours = ["red", "orange", "gold",
           "green", "blue", "purple"]
t.pensize(2)
for i in range(60):
    t.color(colours[i % 6])
    t.forward(i * 3)
    t.left(59)

turtle.done()
```

TURTLE WINDOW





PIP SAYS

i % 6 picks the next colour each time, round and round the list.

A Flower of Circles

MODULE 23

Draw a circle, turn a little, and repeat.

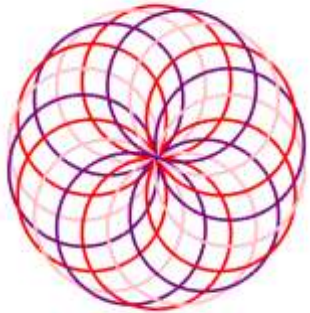
main.py

```
import turtle

t = turtle.Turtle()
t.pensize(2)
petals = ["red", "pink", "purple"]
for i in range(18):
    t.color(petals[i % 3])
    t.circle(70)
    t.left(20)
t.hideturtle()

turtle.done()
```

TURTLE WINDOW





PIP SAYS

18 turns of 20 degrees is 360 degrees, a full turn. hideturtle() hides the arrow.

Quiz Time!

Q1 Which way does the turtle face at the start?

A) Up

B) Right

C) Left

Q2 How many degrees in each turn of a square?

A) 90

B) 60

C) 120

Q3 Which command moves without drawing?

A) `penup()` first

B) `circle()`

C) `color()`

Quiz Time! — Answers

Q1 **Which way does the turtle face at the start?**

A) Up

B) Right

C) Left

Why? The turtle starts in the middle, facing right.

Q2 **How many degrees in each turn of a square?**

A) 90

B) 60

C) 120

Why? 360 ÷ 4 sides = 90 degrees.

Q3 **Which command moves without drawing?**

A) `penup()` first

B) `circle()`

C) `color()`

Why? Lift the pen with penup(), then move.

Module 23 Recap & Challenge

WHAT WE LEARNED

- `import turtle` and `t = turtle.Turtle()`
- `forward()`, `left()` and `right()` move and turn.
- Turn `360 ÷ sides` for a shape.
- `color()`, `pensize()` and fills add style.
- Loops make patterns!

CHALLENGE TIME!

Draw a house: a square for the walls, a triangle on top for the roof, and a filled rectangle for the door. Bonus: add a sun with `circle()`!



MODULE 24 OF 25

24 Bugs and Errors

Every coder makes mistakes. Great coders know how to fix them!

IN THIS MODULE YOU WILL:

- Spot the 3 kinds of bugs
- Read an error message like a pro
- Catch errors with try and except



Three Kinds of Bugs

Bugs are normal. Even expert coders find bugs every single day.

Syntax errors

Spelling or grammar mistakes, like a missing bracket. Python will not start.

Runtime errors

The program starts, then crashes, like dividing by zero.

Logic errors

No crash, but the answer is wrong. The sneakiest bugs!



PIP SAYS

The word **bug** is old. In 1947, engineers found a real moth stuck inside a computer!

How to Read an Error Message

Error messages look scary, but they are Python trying to help. Start at the **bottom**!

ERROR MESSAGE

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(scores[2])
          ~~~~~~^^^
IndexError: list index out of range
```

main.py

```
1 scores = [5, 8]
2 print(scores[2])
```

1. **Which line?** Line 2 of main.py.

2. **The code** that caused the problem.

3. **Read this first!** The error's name and what went wrong.

Common Errors and What They Mean

Error	What it means
`SyntaxError`	Python cannot understand the code. Check brackets, quotes and colons.
`IndentationError`	The spaces at the start of a line are wrong.
`NameError`	That variable or function does not exist. Check the spelling!
`TypeError`	Mixing types that do not fit, like a string + a number.
`ValueError`	The right type but a bad value, like int("cat").
`IndexError`	That list position does not exist.
`KeyError`	That key is not in the dictionary.
`ZeroDivisionError`	Dividing by zero is not allowed.

A Logic Error

The average of 10 and 20 should be 15. But we get 20.0! Why?

main.py

```
1 a = 10
2 b = 20
3 average = a + b / 2
4 print("Average:", average)
```

OUTPUT

```
Average: 20.0
```



PIP SAYS

Division happens before addition. So b / 2 is worked out first! The fix: (a + b) / 2

Detective Tool: Print It Out!

Add extra print() lines to see what your variables are doing.

main.py

```
total = 0
for n in [3, 5, 2]:
    total += n
    print("added", n, "total is now", total)
print("Final:", total)
```

OUTPUT

```
added 3 total is now 3
added 5 total is now 8
added 2 total is now 10
Final: 10
```

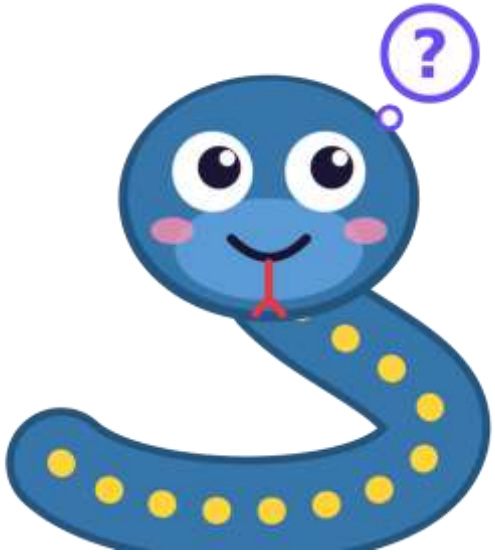


PIP SAYS

When you have found the bug, delete the extra prints.

Bug-Busting Tips

1. Read the **last line** of the error first.
2. Go to the **line number** it tells you. Check the line above too!
3. Check spelling, capitals, brackets, quotes and colons.
4. Explain your code out loud to a toy. This really works! Coders call it **rubber duck debugging**.



Catching Errors: try and except

Instead of crashing, your program can catch an error and keep going.

main.py

```
1 try:
2     number = int("seven")
3 except ValueError:
4     print("Please use digits!")
```

try: Python tries the indented code.

This line causes a ValueError...

...so Python jumps to **except** instead of crashing.

Your friendly message is shown.

try and except in Action

The player typed a word. No crash, just a helpful message!

main.py

```
1 answer = input("How many pets do you have? ")
2 try:
3     pets = int(answer)
4     print("That is", pets * 4, "paws or feet!")
5 except ValueError:
6     print("Oops! Please type a number like 3.")
```

OUTPUT

```
How many pets do you have? three
Oops! Please type a number like 3.
```



Keep Asking Until It Works

Combine while True, try, except and break.

main.py

```
while True:
    try:
        age = int(input("Your age: "))
        break
    except ValueError:
        print("Not a number. Try again!")
print("Thanks! You are", age)
```

OUTPUT

```
Your age: ten
Not a number. Try again!
Your age: 10
Thanks! You are 10
```



PIP SAYS

break only runs if int() worked. Otherwise, except runs and we go round again.

Catching Division by Zero

You can name any kind of error after except.

main.py

```
sweets = 12
friends = 0
try:
    print(sweets / friends)
except ZeroDivisionError:
    print("No friends to share with. More for me!")
```

OUTPUT

```
No friends to share with. More for me!
```

Quiz Time!

Q1 **Which bug gives a wrong answer without crashing?**

A) Syntax error

B) Logic error

C) Runtime error

Q2 **Where should you start reading an error message?**

A) The top line

B) The last line

C) The middle

Q3 **What does except do?**

A) Runs if try had an error

B) Always runs

C) Deletes errors

Quiz Time! — Answers

MODULE 24

Q1 **Which bug gives a wrong answer without crashing?**

A) Syntax error

B) Logic error

C) Runtime error

Why? Logic errors run fine but do the wrong thing.

Q2 **Where should you start reading an error message?**

A) The top line

B) The last line

C) The middle

Why? The last line names the error and explains it.

Q3 **What does except do?**

A) Runs if try had an error

B) Always runs

C) Deletes errors

Why? except only runs when the code in try causes that error.

Module 24 Recap & Challenge

WHAT WE LEARNED

- Syntax, runtime and logic errors.
- Read error messages from the bottom up.
- The line number shows where to look.
- Use print() to watch variables.
- `try` / `except` catches errors without crashing.

CHALLENGE TIME!

Write a calculator that asks for two numbers and divides them. Use try/except to handle words AND dividing by zero!



MODULE 25 OF 25

25 Grand Project: Build a Game

You know so much Python now. Let's use it ALL to build real games!

IN THIS MODULE YOU WILL:

- Plan a game step by step
- Build a Guess the Number game
- Try more projects: a quiz and Rock, Paper, Scissors



The Plan: Guess the Number

Real coders plan first. Break the big job into small steps.

1. The computer picks a secret number from 1 to 20.
2. The player types a guess.
3. Say **Too high!** or **Too low!**
4. Repeat until the guess is right.
5. Say how many guesses it took!

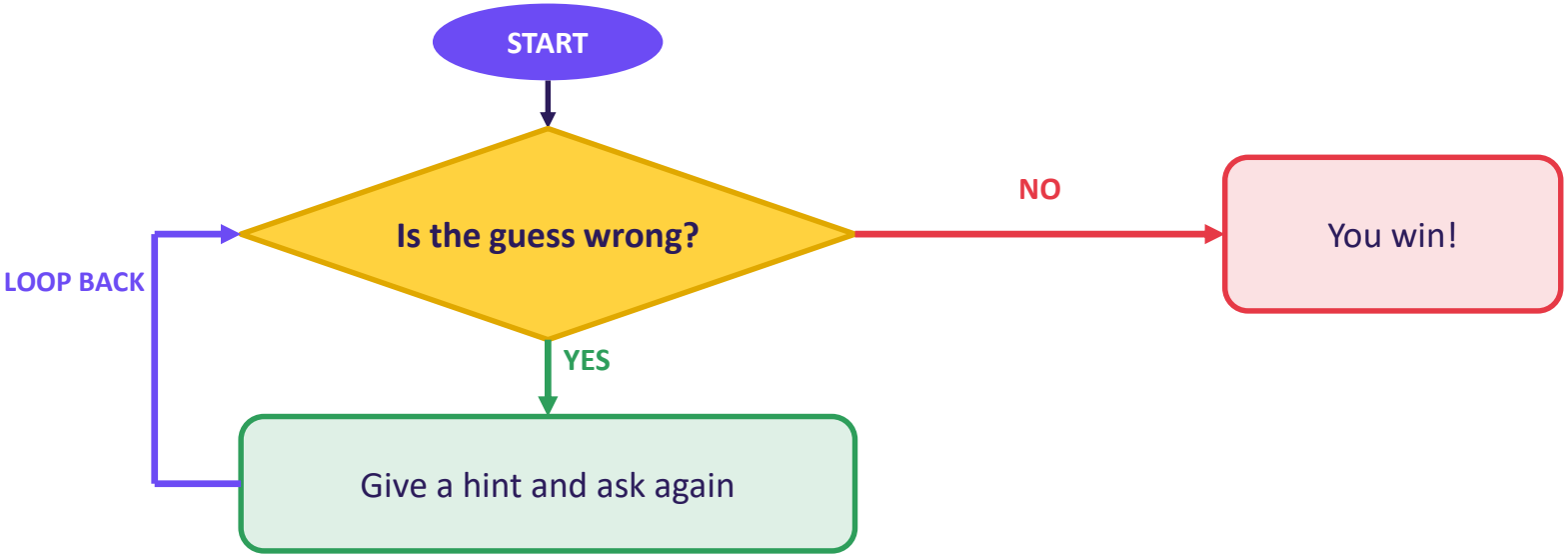


PIP SAYS

We will build it bit by bit, testing as we go. That is how real games are made!

How the Game Flows

This is the main loop of our game.



Step 1: Pick a Secret Number

Start small. Pick the number and ask for one guess.

main.py

```
1 import random
2
3 secret = random.randint(1, 20)
4 guess = int(input("Guess 1 to 20: "))
5 print("You guessed", guess)
6 print("The secret was", secret)
```

OUTPUT

```
Guess 1 to 20: 7
You guessed 7
The secret was 11
```



PIP SAYS

Random numbers change every run, so yours may be different!

Step 2: Give Hints

Add if, elif and else to compare the guess with the secret.

main.py

```
import random

secret = random.randint(1, 20)
guess = int(input("Guess 1 to 20: "))
if guess > secret:
    print("Too high!")
elif guess < secret:
    print("Too low!")
else:
    print("You got it!")
```

OUTPUT

```
Guess 1 to 20: 7
Too low!
```

Step 3: The Full Game!

Add a while loop and a counter. Now it is a real game!

guess.py

```python
import random

secret = random.randint(1, 20)
tries = 0
guess = 0
print("Guess my number (1 to 20)!")
while guess != secret:
    guess = int(input("Your guess: "))
    tries += 1
    if guess > secret:
        print("Too high!")
    elif guess < secret:
        print("Too low!")
print(f"Yes! It was {secret}!")
print(f"You took {tries} tries.")
```

OUTPUT

```
Guess my number (1 to 20)!
Your guess: 10
Too low!
Your guess: 15
Too high!
Your guess: 12
Too high!
Your guess: 11
Yes! It was 11!
You took 4 tries.
```

What Did We Use?

1. **import random** to pick the secret number.
2. **variables** for secret, guess and tries.
3. **input()** and **int()** to get guesses.
4. **while** to keep the game going, and **if / elif** for hints.





PIP SAYS

You used ideas from lots of modules in about 15 lines. Amazing!

Project 2: Quiz Master

A dictionary of questions and answers makes a quiz game.

quiz.py

```
questions = {
    "What is 5 x 6? ": "30",
    "What colour is the sky? ": "blue",
    "How many legs has a spider? ": "8",
}
score = 0
for q, answer in questions.items():
    if input(q).lower() == answer:
        print("Correct!")
        score += 1
    else:
        print("The answer was", answer)
print("Score:", score, "/", len(questions))
```

OUTPUT

```
What is 5 x 6? 30
Correct!
What colour is the sky? Blue
Correct!
How many legs has a spider? 6
The answer was 8
Score: 2 / 3
```

Project 3: Rock, Paper, Scissors

A dictionary remembers what each move beats. random.choice() makes the computer's move.

rps.py

```
import random

beats = {"rock": "scissors",
         "paper": "rock",
         "scissors": "paper"}
player = input("rock, paper or scissors? ")
computer = random.choice(list(beats))
print("Computer chose", computer)
if player == computer:
    print("It is a draw!")
elif beats[player] == computer:
    print("You win!")
else:
    print("Computer wins!")
```

OUTPUT

```
rock, paper or scissors? rock
Computer chose scissors
You win!
```

Make Your Games Even Better

Pick an upgrade and add it to one of your games.



Play again

Wrap the game in a while loop and ask: Play again?



Levels

Easy is 1 to 10, hard is 1 to 100.



Lives

Give the player 5 lives. Lose one for each wrong guess.



No crashes

Use try / except in case the player types a word.



High score

Keep the best score in a variable.



Your idea!

What would make it more fun? Add it!

Final Quiz

Q1 Which module picks the secret number?

- A) `math`
- B) `random`
- C) `time`

Q2 What keeps the game going until the guess is right?

- A) An if
- B) A while loop
- C) A tuple

Q3 What do coders do before building a big program?

- A) Plan it in steps
- B) Type really fast
- C) Skip testing

Final Quiz — Answers

Q1 **Which module picks the secret number?**

A) `math`

B) `random`

C) `time`

Why? random.randint() picks a random whole number.

Q2 **What keeps the game going until the guess is right?**

A) An if

B) A while loop

C) A tuple

Why? while guess != secret keeps looping.

Q3 **What do coders do before building a big program?**

A) Plan it in steps

B) Type really fast

C) Skip testing

Why? Planning small steps, then testing each one, is how real programs are made.

Module 25 Recap & Challenge

WHAT WE LEARNED

- Plan big programs as small steps.
- Build and test one step at a time.
- Games use random, loops, if and variables.
- Dictionaries make great quiz games.
- YOU can build real games in Python!

CHALLENGE TIME!

Upgrade Guess the Number: give the player only 5 tries, add a 'Play again?' question, and use try/except so typing a word does not crash it.



Python Cheat Sheet: The Basics

Code	What it does
`print("Hi")`	Shows Hi on the screen
`# note`	A comment. Python ignores it
`name = "Pip"`	Stores a value in a variable
`input("Name? ")`	Asks a question and waits for an answer
`int("5")  str(5)`	Changes the type of a value
`+  -  *  /  //  %  **`	Maths: add, take, times, divide, whole, remainder, power
`f"Hi {name}"`	Fills in a string with values
`len(x)`	Counts items or characters

Python Cheat Sheet: Decisions and Loops

Code	What it does
`if x > 5:`	Runs the block only if the question is True
`elif x > 2:`	Asks another question
`else:`	Runs if none of the questions were True
`==  !=  <  >  <=  >=`	Compare two values
`and  or  not`	Join or flip True/False questions
`for i in range(5):`	Repeats 5 times (i goes 0 to 4)
`while x < 10:`	Repeats while the question is True
`break  continue`	Leave the loop / skip to the next turn

Python Cheat Sheet: Collections and Functions

Code	What it does
`pets = ["cat", "dog"]`	A list. Change it with .append() and .remove()
`pets[0]`	The first item (indexes start at 0)
`(14, "May")`	A tuple. It cannot change
`{"red", "blue"}`	A set. No repeats allowed
`{"age": 8}`	A dictionary of key: value pairs
`def greet(name):`	Defines your own function
`return x`	Sends a value back from a function
`import random`	Loads a module of extra tools

Python Words Dictionary (A to I)

Argument	A value you send into a function when you call it.
Boolean	A value that is either True or False.
Bug	A mistake in a program.
Code	Instructions written in a programming language.
Comment	A note after #. Python ignores it.
Condition	A question that is True or False.
Debug	To find and fix bugs.
Dictionary	Stores key: value pairs.
Float	A number with a decimal point, like 2.5.
Function	A named set of instructions you can reuse.
IDLE	The program that comes with Python for writing code.
Index	The position number of an item. It starts at 0.
Indent	Spaces at the start of a line, usually 4.
Integer (int)	A whole number, like 7 or -2.

Python Words Dictionary (I to V)

Input	Information that goes into a program.
List	An ordered collection in [ ].
Loop	Code that repeats.
Module	A toolbox of extra code you can import.
Output	What a program shows or gives back.
Parameter	A named box in a function that receives a value.
Program	A list of instructions for a computer.
Return	Sends a value back from a function.
Set	A collection in { } with no repeats.
Shell	The >>> window where you try one line at a time.
String (str)	Text inside quote marks.
Syntax	The spelling and grammar rules of a language.
Tuple	A collection in ( ) that cannot change.
Variable	A named box that stores a value.

What's Next for You?



Pygame Zero

Make arcade games with pictures, sounds and keyboard controls.



micro:bit

Program a tiny computer with lights and buttons using MicroPython.



Raspberry Pi

A small, cheap computer for coding robots and inventions.



Code Club

Free coding clubs for young people, run by the Raspberry Pi Foundation.



Keep practising

Re-do the challenges. Change the code. Break it and fix it!



Your own ideas

The best project is one YOU dream up. Start small and build!

Certificate of Coding

This is to say that

_____

has finished all 25 modules of

Python Programming for Kids & Early Readers



Signed

Date

You Did It!

You are a Python programmer now. Keep coding, keep creating, and keep having fun!

```
print("Goodbye for now!")
```

